

فاز چهارم پروژه مهندسی نرم افزار: معماری نرم افزار و مقایسه رویکردها
موضوع پروژه: سیستم مدیریت هوشمند انرژی ساختمان ها (هوشمندسازی مصرف)
اعضا: هستی قمشی، تانیا اسدی شه میرزادی، ریحانه استاد غلامی

مقدمه فاز چهارم پروژه:

پس از انجام مراحل شناخت مسئله، تحلیل نیازمندی ها، بررسی سناریوهای عملکرد سیستم و مدل سازی بخش های مختلف پروژه در فازهای پیشین، اکنون نوبت به یکی از مهم ترین مراحل چرخه توسعه نرم افزار، یعنی طراحی معماری سیستم رسیده است. در واقع، اگر فازهای قبلی مشخص می کردند که سیستم چه وظایفی بر عهده دارد و چه نیازهایی باید برآورده شوند، این فاز مشخص می کند که سیستم چگونه باید طراحی شود تا بتواند تمامی این نیازها را با بالاترین سطح کارایی، قابلیت اطمینان و امکان توسعه در آینده پاسخ دهد.

از آنجا که این پروژه یک سیستم کنترلی مبتنی بر IoT است، سرعت پاسخ گویی، دقت تصمیم گیری، قابلیت اطمینان و پایداری سیستم از اهمیت بسیار زیادی برخوردار هستند. سامانه باید بتواند اطلاعات دریافتی از حسگرها را در زمان مناسب پردازش کرده و بدون ایجاد تأخیر غیرضروری، فرمان های کنترلی لازم را برای تجهیزات مختلف صادر کند. همچنین با توجه به اینکه این سیستم در محیط واقعی ساختمان مورد استفاده قرار می گیرد، لازم است در شرایط مختلف نیز عملکرد قابل قبولی داشته باشد و حتی در صورت بروز برخی اختلالات، از ادامه فعالیت بازماند.

از دیدگاه مفهومی، ساختار این پروژه شباهت قابل توجهی به نمونه سیستم پمپ انسولین (Insulin Pump) دارد. در هر دو سیستم، مجموعه ای از حسگرها به صورت پیوسته وضعیت محیط را پایش می کنند، اطلاعات جمع آوری شده به بخش تصمیم گیرنده منتقل می شود، نتایج پردازش شده و در نهایت فرمان مناسب برای عملگرها صادر می شود. در حقیقت، هر دو سامانه بر پایه یک حلقه کنترل بسته عمل می کنند؛ به این معنا که خروجی سیستم همواره بر اساس داده های جدید محیط اصلاح شده و فرآیند کنترل به صورت مداوم تکرار می شود. به همین دلیل، اصول طراحی معماری سیستم های کنترلی که در نمونه پمپ انسولین مطرح شده اند، می توانند به عنوان یک الگوی مناسب برای طراحی معماری این پروژه نیز مورد استفاده قرار گیرند.

از سوی دیگر، شرایط واقعی استفاده از این سامانه نیز در انتخاب معماری مناسب نقش مهمی ایفا می کند. در کشور ما احتمال وقوع شرایطی مانند قطعی برق، ناپایداری شبکه اینترنت یا اختلال در ارتباطات شبکه نسبت به بسیاری از سامانه های صرفاً نرم افزاری بیشتر است. بنابراین معماری انتخاب شده باید به گونه ای باشد که علاوه بر ارائه عملکرد مناسب در شرایط عادی، در مواجهه با چنین شرایطی نیز بتواند تا حد امکان عملکرد ایمن و پایداری از خود نشان دهد. به همین دلیل، در طراحی این پروژه تلاش شده است تا ساختار سیستم به شکلی انتخاب شود که وابستگی آن به ارتباطات خارجی کاهش یافته و در صورت بروز اختلال، بخش های حیاتی سیستم همچنان قادر به ادامه فعالیت باشند.

با توجه به این ملاحظات، فاز چهارم پروژه بر سه محور اصلی متمرکز دارد که هر یک نقش مهمی در شکل‌گیری ساختار نهایی سامانه ایفا می‌کنند:

نخستین محور، طراحی معماری کلان سیستم است. در این بخش، مناسب‌ترین الگوهای معماری با توجه به ویژگی‌های سیستم مدیریت هوشمند انرژی مورد بررسی قرار می‌گیرند و دلایل انتخاب آن‌ها به صورت مستند بیان می‌شود. علاوه بر معرفی ساختار کلی سیستم، نمودارهای معماری، نمودار مؤلفه‌ها (Component Diagram) و نمودار استقرار (Deployment Diagram) نیز ارائه خواهند شد تا نحوه قرارگیری اجزای نرم‌افزاری و سخت‌افزاری و ارتباط میان آن‌ها به صورت دقیق مشخص شود. همچنین تصمیمات مهم معماری که در طول فرآیند طراحی اتخاذ شده‌اند، در قالب اسناد تصمیمات معماری ثبت و تشریح خواهند شد.

محور دوم این فاز به معماری خدمت‌گرا (Service-Oriented Architecture یا SOA) اختصاص دارد. با توجه به تنوع وظایف سیستم، وجود بخش‌های مختلف کنترلی و نیاز به توسعه‌پذیری در آینده، استفاده از رویکرد سرویس‌گرا می‌تواند باعث کاهش وابستگی میان اجزای سیستم، افزایش انعطاف‌پذیری و ساده‌تر شدن توسعه قابلیت‌های جدید شود. در این بخش، سرویس‌های اصلی سیستم شناسایی شده، قراردادهای هر سرویس تعریف می‌شوند و نحوه تعامل سرویس‌ها با یکدیگر نیز از طریق نمودار همکاری سرویس‌ها (Service Collaboration Diagram) مورد بررسی قرار خواهد گرفت. همچنین مزایا و محدودیت‌های استفاده از معماری SOA در این پروژه با توجه به ماهیت یک سیستم کنترلی مبتنی بر اینترنت اشیا تحلیل می‌شود.

سومین محور این فاز، مقایسه تحلیلی رویکردهای سنتی و مدرن در توسعه نرم‌افزار است. یکی از وظایف مهم یک مهندس نرم‌افزار تنها انتخاب یک فناوری یا معماری مناسب نیست، بلکه توانایی ارائه استدلال منطقی و فنی برای این انتخاب نیز اهمیت زیادی دارد. به همین دلیل، در این بخش رویکردهای سنتی و مدرن از جنبه‌هایی مانند فرآیند توسعه، ساختار معماری، شیوه مستندسازی، روش‌های آزمون، نحوه استقرار، مدیریت تغییرات و قابلیت توسعه با یکدیگر مقایسه خواهند شد. در پایان نیز با توجه به ویژگی‌های پروژه حاضر، مزایا و معایب هر رویکرد بررسی شده و دلایل انتخاب رویکرد نهایی به صورت مستند ارائه می‌شود.

در مجموع، هدف اصلی این فاز آن است که نشان داده شود معماری انتخاب‌شده برای سیستم مدیریت هوشمند انرژی ساختمان‌ها صرفاً بر اساس سلیقه یا ترجیح شخصی انتخاب نشده، بلکه نتیجه بررسی ویژگی‌های پروژه، تحلیل نیازمندی‌ها، مطالعه معماری‌های مختلف و ارزیابی شرایط واقعی بهره‌برداری از سیستم است. انتظار می‌رود معماری پیشنهادی بتواند علاوه بر تأمین نیازهای فعلی سامانه، زمینه لازم برای توسعه قابلیت‌های جدید، افزایش قابلیت اطمینان، تحمل خطا، سهولت نگهداری و پایداری سیستم در شرایط مختلف عملیاتی را نیز فراهم کند و بستری مناسب برای پیاده‌سازی موفق مراحل بعدی پروژه ایجاد نماید.

بخش الف) طراحی معماری

۱- انتخاب و توجیه الگوی معماری (لایه ای، کلاینت-سرور، Pipe & Filter، رویداد محور و غیره):

۱-۱. اهمیت انتخاب معماری در سیستم‌های نرم‌افزاری:

یکی از مهم‌ترین تصمیماتی که در فرآیند توسعه هر نرم‌افزار اتخاذ می‌شود، انتخاب معماری مناسب برای سیستم است. معماری نرم‌افزار تنها نحوه قرارگیری اجزای مختلف سیستم را مشخص نمی‌کند، بلکه نحوه تعامل آن‌ها، شیوه تبادل اطلاعات، میزان وابستگی میان بخش‌های مختلف، قابلیت توسعه در آینده، سهولت نگهداری و حتی عملکرد سیستم در شرایط بحرانی را نیز تحت تأثیر قرار می‌دهد. به همین دلیل، انتخاب معماری را می‌توان یکی از بنیادی‌ترین تصمیمات مهندسی نرم‌افزار دانست.

در بسیاری از پروژه‌های کوچک، ممکن است انتخاب معماری اهمیت چندانی نداشته باشد، اما در سیستم‌هایی که از اجزای متعددی تشکیل شده‌اند و هم‌زمان با سخت‌افزار، تجهیزات جانبی، پایگاه داده، شبکه و کاربران مختلف در ارتباط هستند، انتخاب یک معماری مناسب اهمیت بسیار بیشتری پیدا می‌کند. در چنین شرایطی، معماری باید علاوه بر پاسخ‌گویی به نیازهای فعلی سیستم، امکان توسعه قابلیت‌های جدید، مدیریت تغییرات و نگهداری آسان‌تر سیستم را نیز فراهم کند. پروژه حاضر، یعنی سیستم مدیریت هوشمند انرژی ساختمان‌ها (هوشمندسازی مصرف)، نمونه‌ای از چنین سیستم‌هایی است. این سامانه علاوه بر آنکه باید داده‌های دریافتی از حسگرهای مختلف را دریافت و تحلیل کند، وظیفه کنترل تجهیزات متعددی مانند سیستم روشنایی، تجهیزات سرمایشی و گرمایشی، موتورخانه و سایر تجهیزات مصرف‌کننده انرژی را نیز بر عهده دارد. از سوی دیگر، به دلیل ماهیت کنترلی سیستم، بسیاری از تصمیمات باید در مدت زمان کوتاهی اتخاذ شوند تا عملکرد سیستم دچار اختلال نشود. بنابراین انتخاب معماری مناسب، تأثیر مستقیمی بر کیفیت عملکرد کل سامانه خواهد داشت.

۱-۲. تحلیل ساختار معماری و الگوهای کاندید:

در مهندسی نرم‌افزار، انتخاب الگو یا سبک معماری یک تصمیم ساده نیست، بلکه پاسخی مستقیم به خصوصیات کیفی یا همان نیازمندی‌های غیرکارکردی (NFRs) سیستم است. سیستم مدیریت هوشمند انرژی ساختمان، به دلیل تعامل با تجهیزات فیزیکی لایه سخت‌افزار و لزوم پردازش بلادرنگ داده‌های حسگرها، یک سیستم سایبر-فیزیکی و یک سیستم کنترلی زمان واقعی محسوب می‌شود. علاوه بر این، شرایط واقعی بهره‌برداری از چنین سامانه‌ای نیز در انتخاب معماری نقش تعیین‌کننده‌ای دارد. با توجه به احتمال وقوع قطعی برق، ناپایداری اینترنت و اختلال در شبکه‌های ارتباطی، معماری سیستم باید به گونه‌ای طراحی شود که حتی در صورت بروز چنین مشکلاتی نیز بتواند عملکرد اصلی خود را حفظ کرده و از توقف کامل فرآیند کنترل جلوگیری کند.

بر همین اساس، پیش از انتخاب معماری نهایی، چند الگوی متداول و شناخته‌شده معماری نرم‌افزار مورد بررسی قرار گرفته‌اند تا میزان تطابق هر یک با نیازهای پروژه مشخص شود. در ادامه، هر یک از این الگوها به صورت جداگانه معرفی و سپس مزایا و محدودیت‌های آن‌ها در این پروژه تحلیل می‌کنیم.

۱-۳. بررسی و تحلیل الگوهای معماری کاندید:

۱-۳-۱. الگوی معماری لایه‌ای:

معماری لایه‌ای یکی از قدیمی‌ترین و پرکاربردترین الگوهای معماری در مهندسی نرم‌افزار است. در این الگو، سیستم به چند لایه مستقل تقسیم می‌شود که هر لایه مسئول انجام مجموعه مشخصی از وظایف است. معمولاً لایه نمایش، لایه منطق کسب‌وکار، لایه دسترسی به داده و لایه زیرساخت به صورت مجزا طراحی می‌شوند و هر لایه تنها با لایه مجاور خود در ارتباط است. این ساختار باعث می‌شود مسئولیت‌های سیستم به صورت شفاف از یکدیگر جدا شده و تغییر در یک بخش، کمترین تأثیر را بر سایر بخش‌ها داشته باشد. یکی از مهم‌ترین مزایای این معماری، افزایش قابلیت نگهداری و توسعه نرم‌افزار است. زمانی که وظایف سیستم در لایه‌های مجزا قرار می‌گیرند، توسعه‌دهندگان می‌توانند بدون ایجاد تغییر در کل سیستم، تنها بخش موردنظر خود را اصلاح یا توسعه دهند. همین ویژگی باعث شده است که معماری لایه‌ای در بسیاری از سامانه‌های سازمانی و پروژه‌های بزرگ مورد استفاده قرار گیرد.

مزایا برای سیستم ما:

جداسازی کامل لایه سخت‌افزار (سنسورها و عملگرها) از لایه منطق محاسباتی (موتور قوانین) و لایه نمایش (داشبورد)، مهم‌ترین مزیت این معماری برای پروژه حاضر محسوب می‌شود. این تفکیک وظایف موجب می‌شود نیازمندی NFR-۰۲ (نیازمندی غیرکارکردی که در فاز یک پروژه به آن اشاره شد) به بهترین شکل تأمین شود. برای مثال، اگر در آینده یک حسگر جدید جایگزین نمونه قبلی شود یا به دلیل محدودیت‌های واردات، مدل متفاوتی از سنسورها مورد استفاده قرار گیرد، تنها لایه مربوط به ارتباط با سخت‌افزار تغییر خواهد کرد و نیازی به اعمال تغییر در موتور قوانین، پایگاه داده یا داشبورد مدیریتی نخواهد بود.

معایب برای سیستم ما:

با وجود مزایای فراوان، معماری لایه‌ای به تنهایی پاسخ‌گوی تمامی نیازهای این پروژه نیست. در معماری لایه‌ای سنتی، بیشتر ارتباطات به صورت همگام انجام می‌شوند. در حالی که در یک سامانه مبتنی بر اینترنت اشیا، حجم زیادی از داده‌های سنسورها به صورت هم‌زمان وارد سیستم می‌شود و این موضوع ممکن است باعث افزایش سربار پردازشی و کاهش سرعت پاسخ‌گویی شود. بنابراین اگرچه معماری لایه‌ای پایه مناسبی برای سازمان‌دهی اجزای سیستم فراهم می‌کند، اما لازم است در کنار سایر الگوهای معماری مورد استفاده قرار گیرد.

۱-۳-۲. الگوی معماری کلاینت-سرور:

معماری کلاینت-سرور یکی از رایج‌ترین الگوهای معماری در سامانه‌های تحت شبکه است. در این الگو، وظیفه ارائه خدمات و مدیریت اطلاعات بر عهده سرور قرار دارد و کلاینت‌ها از طریق شبکه درخواست‌های خود را برای سرور ارسال کرده و پاسخ مناسب را دریافت می‌کنند. این معماری سال‌هاست که در بسیاری از سامانه‌های تحت وب، نرم‌افزارهای سازمانی و سیستم‌های مدیریتی مورد استفاده قرار می‌گیرد. در این پروژه نیز وجود داشبورد مدیریتی، پنل کاربران و امکان مشاهده اطلاعات ساختمان باعث می‌شود این الگو تا حدی قابل استفاده باشد.

مزایا برای سیستم ما:

تمرکز اطلاعات در یک سرور مرکزی، مدیریت ساده‌تر داده‌ها، امکان تهیه نسخه پشتیبان، ارائه گزارش‌های مدیریتی و پشتیبانی مناسب از FR-۰۶ از جمله مهم‌ترین مزایای این معماری محسوب می‌شود. همچنین مدیر ساختمان می‌تواند بدون مراجعه به تجهیزات مختلف، وضعیت تمامی بخش‌ها را از طریق یک داشبورد واحد مشاهده و مدیریت کند. معایب و چالش‌های اساسی:

اعتماد کامل به این معماری برای یک سیستم کنترلی IoT در شرایط زیرساختی کشور، ریسک قابل توجهی ایجاد می‌کند. اگر ارتباط اینترنت یا شبکه دچار اختلال شود، تجهیزات محلی قادر به دریافت دستورات کنترلی نخواهند بود و فرآیند کنترل خودکار تجهیزات (FR-۰۴) متوقف می‌شود. به همین دلیل، استفاده از معماری کلاينت-سرور به صورت خالص نمی‌تواند پاسخ‌گوی تمامی نیازهای پروژه باشد.

۳-۱. الگوی معماری لوله و فیلتر (Pipe and Filter):

معماری لوله و فیلتر (Pipe and Filter) یکی از الگوهای شناخته‌شده در سیستم‌هایی است که حجم زیادی از داده‌ها به صورت پیوسته وارد سیستم شده و قبل از استفاده باید چندین مرحله پردازش روی آن‌ها انجام شود. در این الگو، داده‌ها از طریق مسیرهایی که «لوله» (Pipe) نامیده می‌شوند، میان بخش‌های مختلف سیستم جابه‌جا می‌شوند و هر بخش پردازشی که «فیلتر» (Filter) نام دارد، تنها مسئول انجام یک عملیات مشخص روی داده است. پس از پایان پردازش در هر فیلتر، نتیجه به فیلتر بعدی ارسال می‌شود تا مرحله بعدی پردازش انجام گیرد. یکی از مهم‌ترین مزایای این معماری، تقسیم فرآیند پردازش داده به چند مرحله مستقل است. این ویژگی باعث می‌شود هر مرحله بدون وابستگی به مراحل دیگر توسعه پیدا کند، آزمایش شود یا در صورت نیاز تغییر یابد. همچنین اگر در آینده روش پردازش داده‌ها تغییر کند، معمولاً تنها کافی است همان فیلتر مربوطه اصلاح شود و سایر بخش‌های سیستم بدون تغییر باقی بمانند. در سیستم مدیریت هوشمند انرژی ساختمان، تقریباً تمامی اطلاعات اولیه از طریق حسگرهای مختلف دریافت می‌شود. این اطلاعات ممکن است شامل دمای محیط، میزان نور، حضور افراد، مصرف برق تجهیزات، وضعیت موتورخانه، کیفیت انرژی یا سایر داده‌های مرتبط باشد. بدیهی است که داده‌های خام حسگرها همیشه قابل اعتماد نیستند و ممکن است تحت تأثیر نویز، خطاهای اندازه‌گیری یا شرایط محیطی قرار بگیرند. بنابراین استفاده مستقیم از این داده‌ها برای تصمیم‌گیری می‌تواند باعث صدور فرمان‌های اشتباه شود. مزایا برای سیستم ما:

این الگو برای نیازمندی FR-۰۲ (فراخوانی، اعتبارسنجی و پیش‌پردازش داده‌های سنسوری) گزینه‌ای بسیار مناسب محسوب می‌شود. داده خام تولیدشده توسط حسگرها ابتدا وارد اولین فیلتر می‌شود. در این مرحله داده‌های ناقص یا غیرقابل خواندن شناسایی شده و حذف می‌شوند. سپس داده وارد فیلتر دوم شده و نویزهای احتمالی ناشی از نوسانات لحظه‌ای حسگرها کاهش پیدا می‌کند. در ادامه، داده‌های غیرمنطقی یا خارج از محدوده مجاز بررسی شده و در صورت نیاز اصلاح یا حذف می‌شوند. در نهایت، اطلاعات پالایش‌شده در اختیار موتور قوانین (Rule Engine) قرار می‌گیرد تا فرآیند تصمیم‌گیری بر اساس داده‌های معتبر انجام شود. برای نمونه، ممکن است سنسور شدت نور به دلیل عبور لحظه‌ای یک سایه یا انعکاس نور،

مقدار غیرواقعی ثبت کند. اگر این داده مستقیماً وارد موتور قوانین شود، احتمال دارد سیستم به اشتباه فرمان روشن یا خاموش شدن چراغ‌ها را صادر کند. اما با استفاده از چند مرحله فیلترینگ، چنین داده‌هایی پیش از رسیدن به بخش تصمیم‌گیری اصلاح شده و از ایجاد رفتارهای نادرست جلوگیری می‌شود. علاوه بر این، استفاده از این معماری باعث افزایش خوانایی فرآیند پردازش اطلاعات می‌شود؛ زیرا هر فیلتر تنها یک وظیفه مشخص دارد و مسئولیت آن کاملاً شفاف است. این موضوع فرآیند توسعه، آزمون و نگهداری سیستم را نیز ساده‌تر می‌کند.

معایب برای سیستم ما:

با وجود مزایای قابل توجه، معماری Pipe and Filter به تنهایی نمی‌تواند تمامی نیازهای این پروژه را پوشش دهد. این الگو بیشتر برای پردازش جریان داده طراحی شده است و برای مدیریت تعاملات پیچیده میان اجزای مختلف سیستم مناسب نیست. در پروژه حاضر، علاوه بر پردازش اطلاعات، لازم است میان بخش‌های مختلف مانند سیستم سرمایش، گرمایش، روشنایی، موتورخانه و سایر تجهیزات هماهنگی ایجاد شود. این تعاملات معمولاً نیازمند تبادل پیام، تصمیم‌گیری مشترک و کنترل هم‌زمان چند تجهیز هستند که خارج از محدوده وظایف معماری Pipe and Filter قرار می‌گیرد. به همین دلیل، از این الگو تنها در بخش پردازش اولیه داده‌های حسگرها استفاده می‌شود و سایر وظایف سیستم توسط الگوهای معماری دیگر مدیریت خواهند شد.

۳-۴-۱. الگوی معماری رویدادمحور:

یکی دیگر از الگوهای بسیار مهم در طراحی سیستم‌های مبتنی بر اینترنت اشیا، معماری رویدادمحور است. در این معماری، اجزای مختلف سیستم به جای برقراری ارتباط مستقیم و دائمی با یکدیگر، از طریق رویدادها با هم تعامل می‌کنند. هر زمان اتفاق مشخصی در سیستم رخ دهد، یک رویداد تولید می‌شود و بخش‌هایی که مسئول رسیدگی به آن رویداد هستند، بدون وابستگی مستقیم به تولیدکننده رویداد، عملیات مورد نیاز را انجام می‌دهند. در سال‌های اخیر، این معماری به یکی از پرکاربردترین سبک‌های طراحی برای سامانه‌های هوشمند، سیستم‌های بلادرنگ، اینترنت اشیا و سامانه‌های مانیتورینگ تبدیل شده است؛ زیرا امکان واکنش سریع به تغییرات محیط را فراهم می‌کند و وابستگی میان اجزای مختلف سیستم را تا حد زیادی کاهش می‌دهد. ماهیت پروژه حاضر نیز کاملاً بر پایه وقوع رویدادهای مختلف بنا شده است. تقریباً تمامی تصمیمات سیستم در پاسخ به یک تغییر در محیط اتخاذ می‌شوند. ورود یا خروج افراد از یک فضا، افزایش یا کاهش دمای محیط، تغییر شدت نور، افزایش مصرف انرژی، قطع ارتباط یک حسگر، قطع برق یا حتی بازگشت مجدد برق، همگی نمونه‌هایی از رویدادهایی هستند که باید در سریع‌ترین زمان ممکن توسط سیستم تشخیص داده شده و پاسخ مناسب برای آن‌ها صادر شود.

مزایا برای سیستم ما:

مهم‌ترین مزیت این معماری، افزایش سرعت پاسخ‌گویی سیستم است. برخلاف معماری‌های سنتی که در آن‌ها اجزای سیستم به صورت مداوم وضعیت یکدیگر را بررسی می‌کنند، در معماری رویدادمحور تنها هنگام وقوع یک رویداد عملیات لازم انجام می‌شود. این موضوع باعث کاهش بار پردازشی سیستم و افزایش بهره‌وری منابع می‌شود.

معایب برای سیستم ما:

با وجود مزایای فراوان، استفاده از معماری رویدادمحور پیچیدگی طراحی سیستم را افزایش می‌دهد. مدیریت تعداد زیاد رویدادها، جلوگیری از پردازش تکراری آن‌ها، کنترل ترتیب اجرای عملیات و مدیریت خطاها از جمله چالش‌های این معماری هستند. همچنین در پروژه‌های بزرگ، اگر مدیریت رویدادها به درستی انجام نشود، فرآیند عیب‌یابی سیستم دشوار خواهد شد. با این حال، با توجه به ماهیت کنترلی و بلادرنگ پروژه حاضر، مزایای استفاده از این معماری به مراتب بیشتر از محدودیت‌های آن بوده و استفاده از آن کاملاً توجیه‌پذیر است.

۱-۴. انتخاب معماری نهایی و توجیه مهندسی آن:

پس از بررسی و تحلیل الگوهای مختلف معماری، مشخص شد که هیچ‌یک از آن‌ها به تنهایی قادر به پوشش تمامی نیازهای عملکردی و غیرعملکردی سیستم مدیریت هوشمند انرژی ساختمان نیستند. هر الگو بخشی از نیازهای پروژه را به بهترین شکل برآورده می‌کند، اما در برخی بخش‌ها نیز دارای محدودیت‌هایی است. به همین دلیل، معماری نهایی پروژه به صورت یک معماری تلفیقی طراحی شده است تا نقاط قوت چند الگوی مختلف به صورت هم‌زمان مورد استفاده قرار گیرد. در این پروژه، معماری انتخاب‌شده ترکیبی از معماری لایه‌ای، معماری رویدادمحور و معماری Pipe and Filter است و منطق اصلی آن بر پایه مفهوم Local-First Design و استفاده از Edge Computing شکل گرفته است. در این رویکرد، بخش عمده پردازش‌ها در داخل ساختمان و توسط Edge Gateway انجام می‌شود و وابستگی سیستم به اینترنت تا حد امکان کاهش پیدا می‌کند.

دلایل انتخاب این معماری:

۱. پاسخ به قطعی اینترنت و شبکه (توجیه ۰۱-NFR):

مهم‌ترین دلیل انتخاب این معماری، کاهش وابستگی سیستم به اینترنت است. موتور قوانین، پایگاه داده محلی و فرآیند کنترل تجهیزات همگی داخل Edge Gateway قرار می‌گیرند و در صورت قطع اینترنت نیز به فعالیت خود ادامه می‌دهند. ارتباط با سرور ابری صرفاً برای ذخیره نسخه پشتیبان، گزارش‌گیری و همگام‌سازی اطلاعات انجام می‌شود. بنابراین حتی در صورت قطع کامل اینترنت، فرآیند کنترل تجهیزات متوقف نخواهد شد و داده‌های ثبت‌شده پس از برقراری ارتباط مجدداً با سرور مرکزی همگام‌سازی می‌شوند.

۲. پاسخ به قطعی و وصل ناگهانی برق (توجیه ۰۴-NFR):

با توجه به احتمال قطعی و نوسانات برق، معماری پیشنهادی امکان تعریف رویداد Power_Restored_Event را فراهم می‌کند. پس از بازگشت برق، سیستم مستقیماً تمامی تجهیزات را روشن نمی‌کند، بلکه ابتدا وضعیت پایداری شبکه برق را بررسی کرده و سپس تجهیزات پرمصرف را با فاصله زمانی مناسب و به صورت مرحله‌ای وارد مدار می‌کند. این سیاست علاوه بر افزایش طول عمر تجهیزات، از ایجاد بار ناگهانی روی شبکه داخلی ساختمان جلوگیری خواهد کرد.

۳. افزایش کیفیت داده‌های ورودی:

استفاده از معماری Pipe and Filter در لایه اینترنت اشیا باعث می‌شود داده‌های حسگرها پیش از رسیدن به موتور قوانین، اعتبارسنجی، پالایش و اصلاح شوند. در نتیجه، تصمیمات سیستم بر اساس داده‌های معتبر اتخاذ شده و احتمال صدور فرمان اشتباه به میزان قابل توجهی کاهش می‌یابد.

۴. افزایش توسعه‌پذیری سیستم:

معماری لایه‌ای موجب می‌شود هر بخش سیستم مسئولیت مشخصی داشته باشد و تغییرات آینده با کمترین تأثیر بر سایر بخش‌ها انجام شوند. به عنوان مثال، در صورت اضافه شدن تجهیزات جدید مانند پنل خورشیدی، ایستگاه شارژ خودروهای برقی یا سامانه ذخیره انرژی، تنها کافی است ماژول‌های مربوط به همان تجهیزات توسعه یابند و نیازی به بازطراحی کل سیستم وجود نخواهد داشت.

۵. افزایش سرعت واکنش به تغییرات محیط:

استفاده از معماری رویدادمحور باعث می‌شود سیستم در مواجهه با رویدادهایی مانند تغییر دما، ورود افراد، افزایش مصرف انرژی یا بروز خطا، در کوتاه‌ترین زمان ممکن واکنش مناسب را نشان دهد. این ویژگی یکی از مهم‌ترین نیازهای یک سیستم کنترلی مبتنی بر اینترنت اشیا محسوب می‌شود.

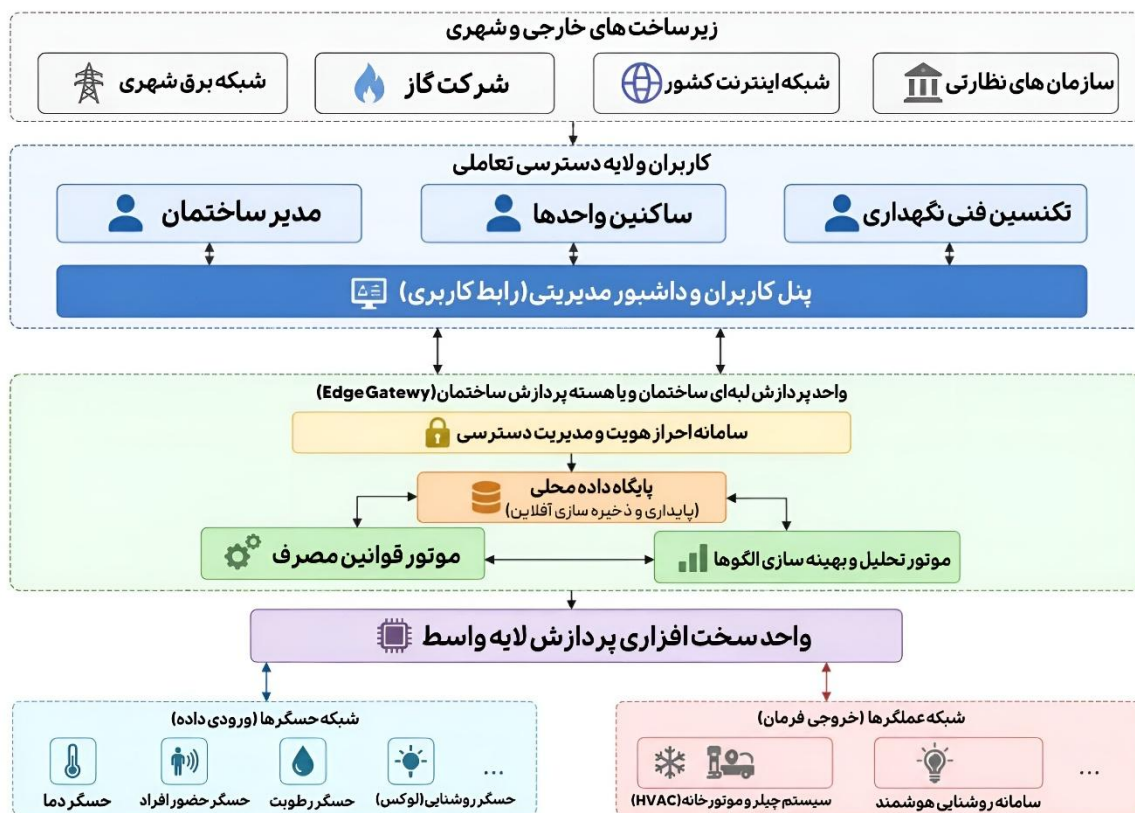
و درنهایت، با توجه به تحلیل انجام شده، می‌توان نتیجه گرفت که استفاده از یک معماری منفرد برای این پروژه پاسخ‌گوی تمامی نیازهای سیستم نخواهد بود. به همین دلیل، معماری تلفیقی انتخاب شده تلاش می‌کند مزایای معماری لایه‌ای، رویدادمحور و Pipe and Filter را در کنار یکدیگر به کار گیرد و در عین حال، محدودیت‌های هر یک را تا حد امکان پوشش دهد. این رویکرد علاوه بر ایجاد ساختاری منسجم و قابل توسعه، امکان عملکرد پایدار سیستم در شرایط واقعی بهره‌برداری، از جمله قطعی اینترنت، نوسانات برق و تغییرات آینده در تجهیزات ساختمان را نیز فراهم می‌کند. از این رو، این معماری مناسب‌ترین گزینه برای پیاده‌سازی سیستم مدیریت هوشمند انرژی ساختمان‌ها در پروژه حاضر تشخیص داده شده است.

۲- نمودار معماری سیستم شامل اجزا و اتصالات

در این قسمت معماری کلی سیستم مدیریت هوشمند انرژی ساختمان‌ها به صورت نمودار ارائه می‌دهیم. هدف از طراحی این معماری، نمایش نحوه سازمان‌دهی اجزای مختلف سیستم و همچنین مشخص کردن ارتباط میان بخش‌های نرم‌افزاری، سخت‌افزاری و کاربران است. در واقع این نمودار، نمایی کلی از ساختار سیستم را در اختیار قرار می‌دهد و نشان می‌دهد که داده‌ها از زمان تولید توسط حسگرها تا مرحله پردازش، ذخیره‌سازی، تصمیم‌گیری و در نهایت ارسال فرمان به تجهیزات کنترلی، چه مسیری را طی می‌کنند.

پس

نمودار زیر، نمای کلی این معماری را نمایش می‌دهد:



معماری طراحی شده برای این سامانه بر پایه یک ساختار سلسله مراتبی و لایه بندی شده شکل گرفته است. استفاده از چنین ساختاری باعث می شود وظایف مختلف سیستم از یکدیگر تفکیک شوند و هر بخش مسئول انجام وظایف مشخصی باشد. این موضوع علاوه بر افزایش خوانایی معماری، توسعه و نگهداری سیستم را نیز ساده تر می کند. در پایین ترین سطح معماری، محیط فیزیکی ساختمان قرار دارد که اطلاعات اولیه مورد نیاز سیستم از آن استخراج می شود. این اطلاعات توسط مجموعه ای از حسگرهای نصب شده در نقاط مختلف ساختمان جمع آوری می شوند و وضعیت لحظه ای محیط را در اختیار سیستم قرار می دهند. در بالاترین سطح نیز کاربران مختلف قرار دارند که از طریق پنل مدیریتی یا داشبورد سامانه با سیستم تعامل برقرار می کنند. میان این دو بخش، هسته اصلی پردازش سیستم قرار گرفته است که وظیفه دریافت داده ها، پردازش اطلاعات، اجرای قوانین کنترلی و ارسال فرمان به تجهیزات را بر عهده دارد. یکی از ویژگی های مهم این معماری، استقلال نسبی هسته پردازش از شرایط بیرونی است. از آنجا که سیستم در بستر اینترنت اشیا فعالیت می کند، همواره احتمال بروز اختلال در ارتباطات شبکه یا محدودیت های زیرساختی وجود دارد. به همین دلیل، بخش عمده ای از پردازش های اصلی در گیت وی محلی ساختمان انجام می شود تا حتی در شرایطی که ارتباط با اینترنت دچار مشکل شود، فرآیندهای کنترلی و تصمیم گیری های ضروری بدون وقفه ادامه پیدا کنند. این رویکرد موجب افزایش پایداری و قابلیت اطمینان سیستم در شرایط مختلف بهره برداری خواهد شد.

در بخش بالایی معماری، کاربران سیستم قرار دارند که هر یک متناسب با سطح دسترسی خود با سامانه ارتباط برقرار می‌کنند. این کاربران از طریق پنل مدیریتی یا داشبورد تحت وب و یا نرم‌افزار مربوطه قادر خواهند بود اطلاعات مورد نیاز خود را مشاهده کرده و در صورت لزوم برخی تنظیمات را اعمال کنند.

در این معماری سه گروه اصلی از کاربران در نظر گرفته شده‌اند:

مدیر ساختمان مسئول مدیریت کلی سامانه است و می‌تواند تنظیمات اصلی سیستم، سیاست‌های مدیریت انرژی، آستانه‌های مصرف و وضعیت عملکرد تجهیزات را کنترل کند. همچنین فعال‌سازی برخی حالت‌های خاص مانند مدیریت مصرف در ساعات اوج بار نیز از طریق این کاربر انجام می‌شود. ساکنان ساختمان بیشتر نقش کاربران نهایی را ایفا می‌کنند و امکان مشاهده میزان مصرف انرژی، دریافت گزارش‌های دوره‌ای و بررسی وضعیت تجهیزات مرتبط با واحد خود را خواهند داشت. سطح دسترسی این کاربران محدود بوده و امکان ایجاد تغییر در تنظیمات حیاتی سیستم برای آن‌ها وجود ندارد. تکنسین نگهداری نیز مسئول پایش وضعیت تجهیزات، بررسی هشدارهای خرابی و انجام عملیات تعمیر و نگهداری است. در مواقع ضروری، این کاربر می‌تواند از قابلیت Manual Override استفاده کرده و کنترل برخی تجهیزات را به صورت دستی در اختیار بگیرد تا از بروز اختلال در عملکرد ساختمان جلوگیری شود. تشریح مؤلفه‌های داخلی معماری و وظایف هر بخش:

سامانه احراز هویت و مدیریت دسترسی اولین بخش در مسیر ورود کاربران به سیستم محسوب می‌شود. وظیفه این ماژول، بررسی هویت کاربران و کنترل سطح دسترسی آن‌ها است تا تنها افراد مجاز بتوانند به امکانات مختلف سامانه دسترسی داشته باشند. همچنین با استفاده از مکانیزم کنترل دسترسی مبتنی بر نقش (RBAC)، از اعمال تغییرات غیرمجاز در بخش‌های حساس سیستم جلوگیری می‌شود.

پایگاه داده مرکزی یا محلی به عنوان مخزن اصلی اطلاعات سیستم عمل می‌کند. تمامی داده‌های مربوط به مصرف انرژی، اطلاعات کاربران، وضعیت تجهیزات، سوابق عملکرد سیستم و داده‌های دریافت‌شده از حسگرها به همراه زمان ثبت آن‌ها در این بخش ذخیره می‌شوند. وجود این اطلاعات علاوه بر امکان گزارش‌گیری، زمینه لازم برای تحلیل‌های بعدی و بهینه‌سازی مصرف انرژی را نیز فراهم می‌کند.

موتور قوانین مصرف انرژی یکی از مهم‌ترین بخش‌های سامانه محسوب می‌شود. این ماژول قوانین تعریف‌شده توسط مدیر سیستم را دریافت کرده و به صورت لحظه‌ای شرایط محیط را با این قوانین مقایسه می‌کند. در صورتی که شرایط تعریف‌شده برقرار باشد، موتور قوانین بدون نیاز به دخالت کاربر فرمان مناسب را برای تجهیزات ارسال می‌کند. به عنوان نمونه، اگر حسگر حضور عدم حضور افراد را در یک بخش از ساختمان تشخیص دهد، این موتور می‌تواند سیستم سرمایش یا روشنایی آن قسمت را به حالت کم مصرف تغییر دهد.

موتور تحلیل و بهینه‌سازی مصرف انرژی وظیفه بررسی اطلاعات ذخیره‌شده در پایگاه داده را بر عهده دارد. این بخش با تحلیل الگوهای مصرف در بازه‌های زمانی مختلف، رفتار مصرف‌کنندگان را بررسی کرده و در صورت مشاهده افزایش غیرعادی مصرف یا تغییر الگوهای معمول، پیشنهاد‌های لازم را برای بهینه‌سازی ارائه می‌دهد یا در صورت نیاز برخی

پارامترهای کنترلی را به صورت خودکار اصلاح می‌کند.

واحد پردازش لبه و لایه واسط سخت‌افزار به عنوان پل ارتباطی میان تجهیزات فیزیکی و بخش نرم‌افزاری سیستم عمل می‌کند. این بخش اطلاعات دریافتی از انواع حسگرها را دریافت کرده، آن‌ها را به قالبی یکنواخت تبدیل می‌کند و سپس برای پردازش در اختیار سایر بخش‌های نرم‌افزار قرار می‌دهد. همچنین فرمان‌های صادرشده توسط موتور قوانین را نیز به زبان قابل فهم تجهیزات سخت‌افزاری تبدیل کرده و برای عملگرها ارسال می‌کند. استفاده از این لایه باعث می‌شود وابستگی سیستم به نوع خاصی از پروتکل‌های ارتباطی کاهش یافته و امکان استفاده همزمان از فناوری‌هایی مانند Modbus، Zigbee و Wi-Fi فراهم شود.

و در نهایت تحلیل مسیر جریان اطلاعات در معماری سیستم:

بررسی نمودار معماری نشان می‌دهد که اطلاعات در این سامانه در قالب سه جریان اصلی میان اجزای مختلف جابه‌جا می‌شوند که هر یک نقش مشخصی در عملکرد سیستم دارند. جریان اول، جریان اطلاعات بالارونده (Sensor to Core) است. در این مسیر، حسگرهای مختلف مانند حسگر دما، رطوبت، نور و حضور، اطلاعات محیط را به صورت دوره‌ای یا بر اساس وقوع رویداد برای واحد پردازش لبه ارسال می‌کنند. پس از انجام بررسی‌های اولیه و اعتبارسنجی داده‌ها، اطلاعات هم‌زمان در اختیار پایگاه داده و موتور قوانین قرار می‌گیرد تا علاوه بر ثبت سوابق، تصمیم‌گیری لحظه‌ای نیز امکان‌پذیر باشد.

جریان دوم، جریان اطلاعات پایین‌رونده (Core to Actuator) است. پس از تحلیل داده‌ها و اجرای قوانین کنترلی، دستورات لازم از هسته پردازش به سمت عملگرهای سیستم ارسال می‌شود. این عملگرها می‌توانند شامل تجهیزات سرمایشی و گرمایشی، سیستم روشنایی، کلیدهای هوشمند یا سایر تجهیزات کنترلی ساختمان باشند. در نتیجه اجرای این دستورات، وضعیت فیزیکی محیط متناسب با شرایط موجود تغییر می‌کند و اهداف مدیریت انرژی محقق می‌شود. جریان سوم، جریان اطلاعات نظارتی و هشدار (Management Stream) است. در این مسیر، سیستم به صورت مداوم وضعیت عملکرد تجهیزات و داده‌های دریافتی را پایش می‌کند. در صورت بروز هرگونه خرابی، قطع ارتباط با تجهیزات، عملکرد غیرعادی حسگرها یا افزایش غیرمعتارف مصرف انرژی، اطلاعات به ماژول مدیریت هشدار ارسال می‌شود. این ماژول ضمن ثبت رخداد، پیام هشدار مناسبی را در اختیار کاربران مسئول، به‌ویژه تکنسین نگهداری، قرار می‌دهد تا اقدامات لازم برای رفع مشکل یا اعمال کنترل دستی (Manual Override) انجام شود.

با توجه به معماری ارائه‌شده، تمامی اجزای سیستم در قالب یک ساختار منسجم و هماهنگ با یکدیگر تعامل دارند و هر بخش مسئول انجام وظیفه مشخصی در فرآیند مدیریت هوشمند انرژی است. این معماری علاوه بر فراهم کردن امکان پایش و کنترل لحظه‌ای تجهیزات، زمینه لازم برای ذخیره‌سازی اطلاعات، تحلیل الگوهای مصرف، تصمیم‌گیری خودکار و مدیریت بهینه انرژی را نیز فراهم می‌کند. همچنین تفکیک مناسب وظایف میان مؤلفه‌های مختلف باعث شده است که توسعه، نگهداری و ارتقای سامانه در آینده با سهولت بیشتری انجام شود و عملکرد پایدار سیستم در شرایط مختلف حفظ گردد.

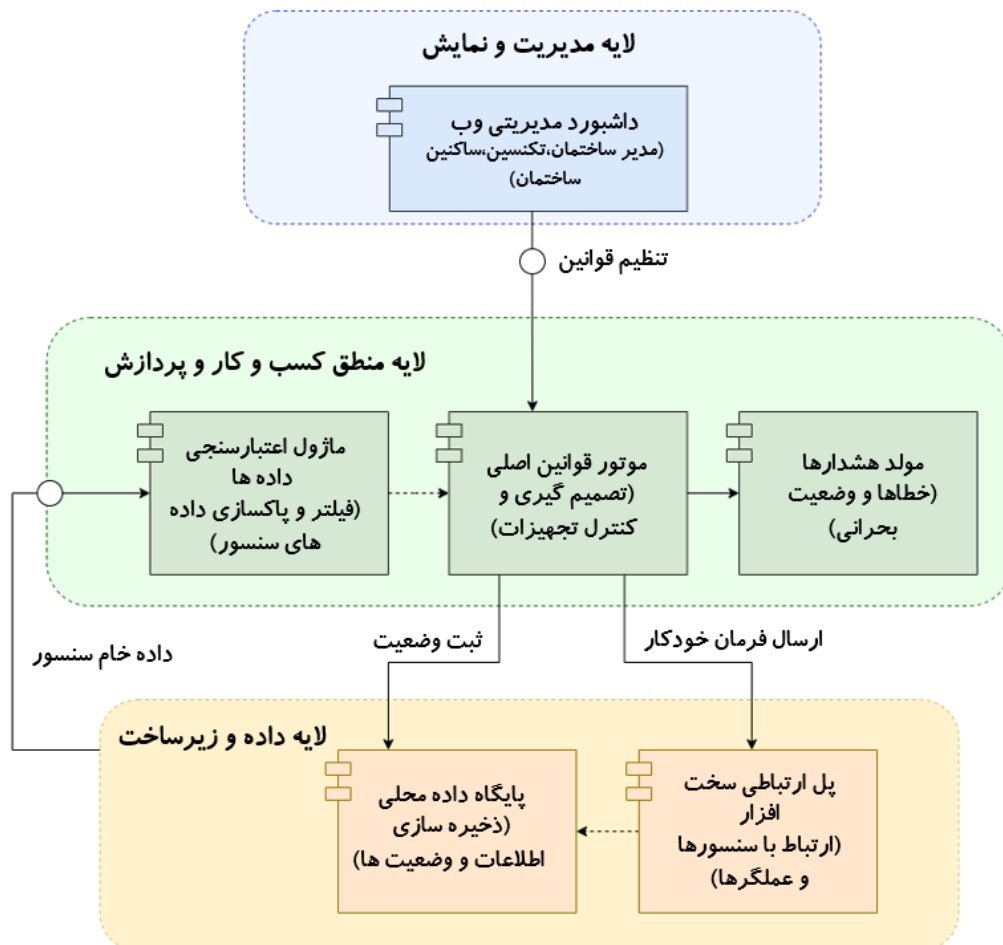
۳- نمودار مؤلفه (Component Diagram) و نمودار استقرار (Deployment Diagram):

نمودار مؤلفه:

پس از تعیین معماری کلی سیستم و مشخص شدن ارتباط میان اجزای اصلی، در این بخش ساختار داخلی نرم افزار از دیدگاه مؤلفه های تشکیل دهنده مورد بررسی قرار می گیرد. نمودار مؤلفه (Component Diagram) یکی از مهم ترین نمودارهای طراحی نرم افزار است که نحوه تقسیم سیستم به ماژول های مستقل، ارتباط میان آن ها و وابستگی های موجود را نمایش می دهد. هدف از طراحی این نمودار، ایجاد ساختاری منظم و ماژولار برای سیستم است تا هر بخش از نرم افزار مسئول انجام وظایف مشخصی باشد و بدون ایجاد وابستگی مستقیم به سایر بخش ها بتواند توسعه یا نگهداری شود.

در سیستم مدیریت هوشمند انرژی ساختمان، با توجه به گستردگی فرآیندهای پردازشی و ارتباط هم زمان با تجهیزات مختلف، استفاده از معماری مبتنی بر مؤلفه اهمیت ویژه ای دارد. در این رویکرد، هر قابلیت اصلی سیستم در قالب یک مؤلفه مستقل پیاده سازی شده است و ارتباط میان مؤلفه ها از طریق واسطه های مشخص و استاندارد انجام می شود. این موضوع علاوه بر افزایش خوانایی ساختار نرم افزار، موجب کاهش پیچیدگی کدها، تسهیل فرآیند توسعه و همچنین افزایش قابلیت نگهداری سیستم در آینده خواهد شد.

نمودار زیر ساختار مؤلفه های نرم افزاری سیستم و نحوه ارتباط میان آن ها را نمایش می دهد:



همان‌طور که در نمودار مؤلفه مشاهده می‌شود، سیستم نرم‌افزاری از مجموعه‌ای از ماژول‌های مستقل تشکیل شده است که هر یک وظیفه مشخصی را بر عهده دارند. این تفکیک وظایف باعث شده است که تغییر یا توسعه هر بخش، کمترین تأثیر را بر سایر قسمت‌های سیستم داشته باشد و فرآیند نگهداری نرم‌افزار در آینده با سهولت بیشتری انجام شود. علاوه بر این، استفاده از مؤلفه‌های مستقل امکان آزمون، اشکال‌زدایی و به‌روزرسانی هر بخش را نیز به صورت مجزا فراهم می‌کند. تحلیل مؤلفه‌های لایه منطق تجاری:

هسته اصلی پردازش سیستم در لایه منطق تجاری قرار دارد. این بخش مسئول دریافت اطلاعات، پردازش داده‌ها، تصمیم‌گیری و ایجاد خروجی مناسب برای سایر قسمت‌های سیستم است. در معماری حاضر، این لایه از چند مؤلفه اصلی تشکیل شده که هر یک نقش مشخصی در فرآیند مدیریت انرژی ایفا می‌کنند.

مؤلفه اعتبارسنجی داده‌ها: اولین مؤلفه موجود در مسیر پردازش اطلاعات، ماژول اعتبارسنجی داده‌ها است. تمامی اطلاعات دریافت‌شده از حسگرهای مختلف ابتدا وارد این بخش می‌شوند تا صحت و اعتبار آن‌ها بررسی شود. در محیط‌های واقعی، ممکن است داده‌های ارسالی توسط سنسورها تحت تأثیر نویز، اختلالات محیطی یا خطاهای ارتباطی قرار گیرند. در چنین شرایطی، اگر داده‌های نادرست مستقیماً وارد فرآیند تصمیم‌گیری شوند، احتمال صدور فرمان‌های اشتباه افزایش خواهد یافت. به همین دلیل، این مؤلفه با استفاده از الگوریتم‌های اعتبارسنجی، حذف داده‌های غیرعادی و فیلتر کردن نویزهای احتمالی، تنها اطلاعات معتبر را برای ادامه پردازش در اختیار سایر بخش‌های سیستم قرار می‌دهد. وجود این ماژول نقش مهمی در افزایش دقت تصمیم‌گیری و جلوگیری از بروز خطاهای کنترلی ایفا می‌کند.

مؤلفه هسته موتور قوانین: پس از اعتبارسنجی اطلاعات، داده‌های معتبر به هسته موتور قوانین منتقل می‌شوند. این مؤلفه مهم‌ترین بخش تصمیم‌گیرنده سیستم محسوب می‌شود و تمامی منطق‌های کنترلی در این قسمت اجرا می‌شوند. موتور قوانین شرایط لحظه‌ای محیط را با قوانین تعریف‌شده توسط مدیر سیستم مقایسه می‌کند و در صورت برقرار بودن شرایط لازم، فرمان مناسب را تولید می‌کند. این مؤلفه به گونه‌ای طراحی شده است که وابستگی مستقیمی به پایگاه داده یا تجهیزات سخت‌افزاری نداشته باشد و تنها بر اساس داده‌های ورودی و قوانین از پیش تعریف‌شده تصمیم‌گیری کند. این استقلال باعث می‌شود توسعه یا اصلاح قوانین کنترلی بدون ایجاد تغییر در سایر بخش‌های نرم‌افزار امکان‌پذیر باشد.

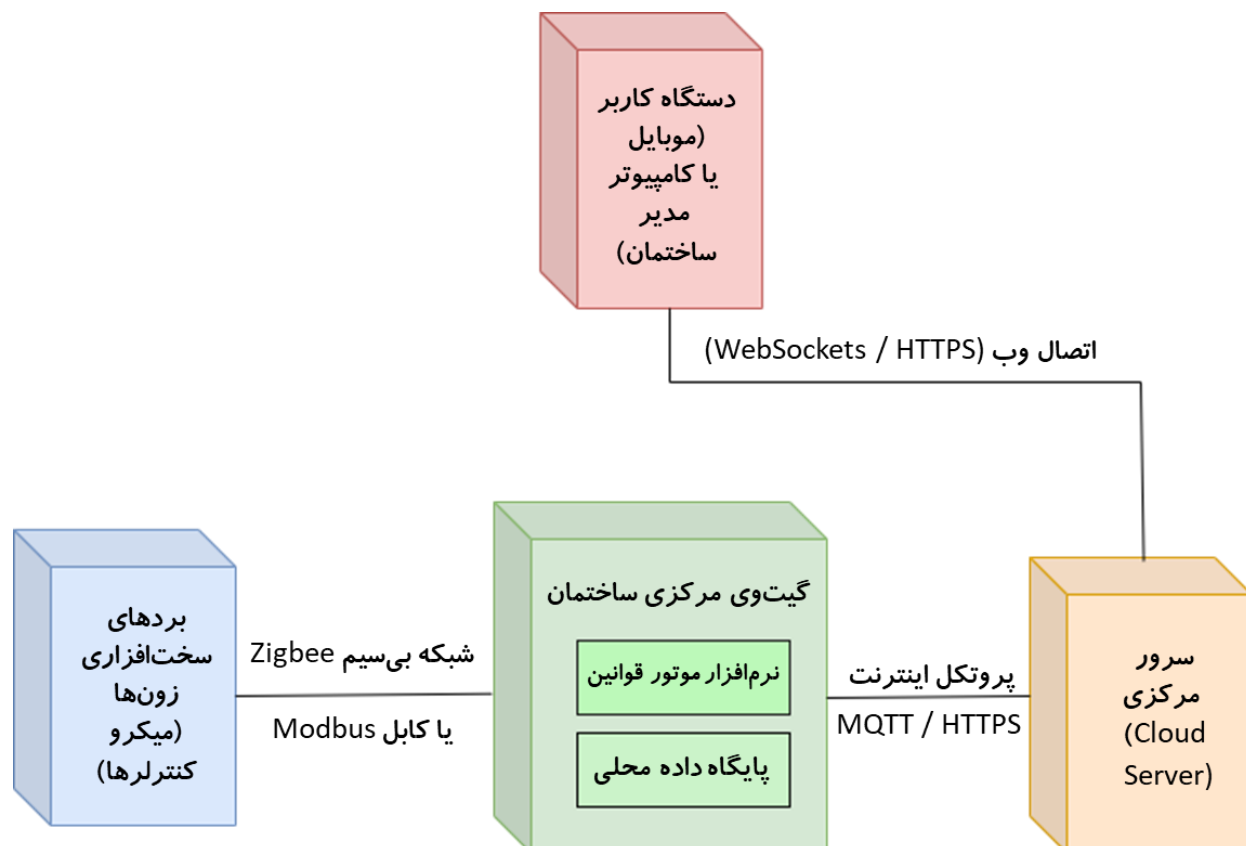
مؤلفه مولد هشدار: یکی دیگر از مؤلفه‌های مهم سیستم، ماژول مدیریت هشدار است. این بخش وظیفه پایش رویدادهای بحرانی و تولید پیام‌های هشدار را بر عهده دارد. هر زمان که ماژول اعتبارسنجی وجود خطا، خرابی تجهیزات یا دریافت داده‌های غیرعادی را تشخیص دهد، اطلاعات مربوط به آن رویداد در اختیار این مؤلفه قرار می‌گیرد. ماژول مدیریت هشدار پس از تحلیل وضعیت، پیام مناسب را تولید کرده و آن را برای داشبورد مدیریتی ارسال می‌کند تا کاربران مسئول از وقوع مشکل مطلع شوند. این فرآیند باعث می‌شود خطاهای احتمالی در کوتاه‌ترین زمان شناسایی شده و اقدامات لازم برای رفع آن‌ها انجام گیرد.

در کل، بررسی نمودار مؤلفه نشان می‌دهد که ساختار نرم‌افزار بر پایه اصل تفکیک مسئولیت‌ها طراحی شده است و هر مؤلفه تنها وظایف مشخصی را بر عهده دارد. استفاده از مؤلفه‌های مستقل و واسط‌های استاندارد، وابستگی میان بخش‌های مختلف را کاهش داده و امکان توسعه، نگهداری، آزمون و به‌روزرسانی سیستم را ساده‌تر کرده است. در نتیجه، معماری

نرم افزار علاوه بر پاسخ گویی به نیازهای فعلی سامانه، از انعطاف لازم برای توسعه قابلیت های جدید و سازگاری با تغییرات آینده نیز برخوردار خواهد بود.

نمودار استقرار:

پس از مشخص شدن ساختار منطقی نرم افزار و مؤلفه های تشکیل دهنده آن، لازم است نحوه استقرار فیزیکی این مؤلفه ها بر روی تجهیزات سخت افزاری نیز مورد بررسی قرار گیرد. نمودار استقرار (Deployment Diagram) یکی از مهم ترین نمودارهای طراحی سیستم است که محل اجرای هر بخش از نرم افزار، ارتباط میان گره های سخت افزاری و نحوه تبادل اطلاعات در سطح شبکه را نمایش می دهد. در سیستم مدیریت هوشمند انرژی ساختمان ها، تنها طراحی بخش نرم افزار برای دستیابی به عملکرد مطلوب کافی نیست، بلکه نحوه توزیع مؤلفه ها روی تجهیزات مختلف نیز تأثیر مستقیمی بر سرعت پردازش، قابلیت اطمینان، توسعه پذیری و پایداری سیستم خواهد داشت. به همین دلیل، در این پروژه از معماری مبتنی بر محاسبات لبه ای (Edge Computing) استفاده شده است تا بخش عمده پردازش ها در نزدیک ترین نقطه به تجهیزات ساختمان انجام شود و وابستگی سیستم به ارتباط دائمی با سرورهای ابری کاهش یابد. نمودار زیر نحوه استقرار فیزیکی اجزای سیستم و ارتباط میان آن ها را نمایش می دهد:



۴- توصیف تصمیمات معماری و Architecture Decision Records:

در طراحی سیستم‌های نرم‌افزاری، به‌ویژه سیستم‌هایی که علاوه بر نرم‌افزار با تجهیزات سخت‌افزاری نیز در ارتباط هستند، تنها ارائه نمودارهای معماری برای توصیف ساختار سیستم کافی نیست. در بسیاری از موارد، مهم‌تر از خود معماری، دلایل انتخاب آن است. به همین دلیل در مهندسی نرم‌افزار از سندی با عنوان Architecture Decision Records (ADR) استفاده می‌شود. این سند مجموعه‌ای از تصمیمات مهم معماری را ثبت می‌کند و توضیح می‌دهد که هر تصمیم بر چه اساسی اتخاذ شده، چه گزینه‌هایی برای آن وجود داشته و در نهایت چه مزایا و محدودیت‌هایی را برای سیستم به همراه خواهد داشت.

در پروژه ما نیز با توجه به اینکه سیستم مدیریت هوشمند انرژی ساختمان‌ها در محیطی مبتنی بر اینترنت اشیا (IoT) فعالیت می‌کند و علاوه بر بخش نرم‌افزار با تجهیزات فیزیکی مختلف در ارتباط است، تصمیمات معماری نقش بسیار مهمی در عملکرد نهایی سامانه دارند. همچنین با در نظر گرفتن شرایط بهره‌برداری از سیستم، محدودیت‌های زیرساختی و نیازمندی‌های استخراج‌شده در فازهای قبل، چند تصمیم مهم معماری اتخاذ شده است که در ادامه به صورت مستند ارائه می‌شوند.

۴-۱. سند تصمیم‌گیری اول (ADR-۰۱):

یکی از مهم‌ترین تصمیمات این پروژه مربوط به محل اجرای منطق اصلی سیستم است. در بسیاری از سامانه‌های اینترنت اشیا، تمامی اطلاعات جمع‌آوری‌شده توسط حسگرها ابتدا به سرورهای ابری ارسال می‌شوند و سپس عملیات تحلیل، تصمیم‌گیری و تولید فرمان در همان سرور انجام می‌شود. اگرچه این روش در بسیاری از کاربردها مناسب است، اما برای سیستم مدیریت هوشمند انرژی ساختمان با چالش‌های قابل توجهی همراه خواهد بود. در چنین ساختاری، عملکرد صحیح سیستم به کیفیت ارتباط اینترنت وابسته است. در صورتی که ارتباط ساختمان با اینترنت دچار اختلال شود یا دسترسی به سرورهای ابری برای مدتی امکان‌پذیر نباشد، فرآیند پردازش اطلاعات نیز متوقف شده و تجهیزات کنترلی قادر به دریافت فرمان‌های جدید نخواهند بود. در نتیجه ممکن است سیستم سرمایش، گرمایش، روشنایی یا سایر تجهیزات هوشمند برای مدت قابل توجهی بدون مدیریت باقی بمانند. از آنجا که یکی از اهداف اصلی این پروژه افزایش پایداری و قابلیت اطمینان سیستم است، لازم بود معماری به گونه‌ای انتخاب شود که وابستگی مستقیم به اینترنت تا حد امکان کاهش یابد و فرآیند کنترل تجهیزات حتی در شرایط قطع ارتباط نیز ادامه پیدا کند.

پیش از انتخاب راهکار نهایی، دو گزینه اصلی مورد بررسی قرار گرفت:

گزینه اول پردازش کامل اطلاعات در سرور ابری بود. در این روش تمامی داده‌های حسگرها به فضای ابری ارسال شده و تصمیمات کنترلی نیز در همان محیط اتخاذ می‌شود. هرچند این روش مدیریت متمرکز اطلاعات را ساده‌تر می‌کند، اما وابستگی بسیار زیادی به اینترنت ایجاد خواهد کرد.

گزینه دوم انتقال هسته پردازش به گیت‌وی محلی ساختمان بود. در این روش، تصمیمات اصلی در داخل ساختمان گرفته می‌شوند و سرور ابری تنها وظیفه ذخیره‌سازی اطلاعات، تهیه نسخه پشتیبان و ارائه گزارش‌های مدیریتی را بر عهده خواهد داشت.

پس از بررسی مزایا و محدودیت‌های هر دو گزینه، گزینه دوم به عنوان معماری نهایی انتخاب شد.

تصمیم گرفته شده (Decision): بر اساس این تصمیم، موتور قوانین (Rule Engine)، پایگاه داده عملیاتی و بخش اصلی پردازش اطلاعات روی گیت‌وی محلی ساختمان مستقر می‌شوند. این گیت‌وی به عنوان هسته اصلی سیستم عمل کرده و تمامی اطلاعات دریافتی از حسگرها را پردازش می‌کند. در ادامه، تنها داده‌های مورد نیاز برای آرشیو، گزارش‌گیری و تحلیل‌های بلندمدت به سرور ابری منتقل می‌شوند. به این ترتیب، حتی در صورت قطع کامل ارتباط اینترنت، سیستم همچنان قادر خواهد بود اطلاعات محیط را دریافت کرده، قوانین کنترلی را اجرا کند و فرمان‌های لازم را برای تجهیزات مختلف صادر نماید.

مهم‌ترین دلیل انتخاب این معماری، افزایش قابلیت اطمینان سیستم در شرایط واقعی بهره‌برداری است. از آنجا که مدیریت انرژی ساختمان باید به صورت پیوسته انجام شود، توقف سیستم به دلیل اختلال در اینترنت قابل قبول نیست. همچنین انجام پردازش‌ها در نزدیک‌ترین نقطه به تجهیزات باعث کاهش زمان پاسخ‌گویی سیستم شده و سرعت اجرای فرمان‌های کنترلی را افزایش می‌دهد. از سوی دیگر، نگهداری اطلاعات حساس ساختمان در پایگاه داده محلی نیز می‌تواند امنیت بیشتری نسبت به ارسال مداوم تمامی داده‌ها به فضای ابری ایجاد کند.

پیامدهای تصمیم شامل مزایا و معایب می‌باشد که مزایای آن:

- کاهش وابستگی سیستم به اینترنت
- افزایش پایداری عملکرد سامانه
- کاهش زمان پاسخ‌گویی در اجرای فرمان‌های کنترلی
- افزایش امنیت اطلاعات داخلی ساختمان
- ادامه عملکرد سیستم حتی در صورت اختلال ارتباطات خارجی و معایب آن:
- نیاز به استفاده از گیت‌وی با توان پردازشی مناسب
- افزایش هزینه اولیه پیاده‌سازی

پیچیده‌تر شدن فرآیند همگام‌سازی اطلاعات میان گیت‌وی و سرور Cloud

۴-۲. سند تصمیم‌گیری دوم (ADR-۰۲):

یکی از شرایطی که در ساختمان‌های هوشمند ممکن است رخ دهد، قطع و وصل ناگهانی برق است. اگر پس از بازگشت جریان برق تمامی تجهیزات پرمصرف به صورت هم‌زمان فعال شوند، جریان اولیه مصرف می‌تواند بسیار زیاد باشد و به تجهیزات الکتریکی ساختمان آسیب وارد کند. در چنین شرایطی علاوه بر افزایش احتمال قطع مجدد برق، امکان آسیب دیدن برخی تجهیزات حساس مانند چیلرها، پمپ‌ها، تابلوهای برق و تجهیزات کنترلی نیز وجود دارد. بنابراین لازم بود مکانیزمی در معماری سیستم پیش‌بینی شود که فرآیند راه‌اندازی تجهیزات را به صورت کنترل‌شده مدیریت کند.

تصمیم گرفته شده (Decision): بر اساس این تصمیم، پس از بازگشت برق، سیستم ابتدا در وضعیت Safe Mode راه‌اندازی می‌شود. در این وضعیت، تجهیزات پرمصرف تا زمان بررسی شرایط شبکه برق غیرفعال باقی می‌مانند. پس از بررسی پایداری ولتاژ و اطمینان از مناسب بودن شرایط، موتور قوانین تجهیزات را به صورت مرحله‌ای و با فاصله زمانی مشخص فعال می‌کند.

برای مثال ابتدا پمپ‌ها راه‌اندازی می‌شوند، سپس پس از گذشت چند دقیقه سیستم سرمایشی فعال شده و در نهایت سایر تجهیزات روشن خواهند شد. هدف اصلی این تصمیم، محافظت از تجهیزات ساختمان و جلوگیری از وارد شدن شوک الکتریکی به شبکه داخلی است. علاوه بر این، راه‌اندازی مرحله‌ای تجهیزات باعث توزیع بهتر بار مصرفی شده و احتمال بروز خطاهای ناشی از افزایش ناگهانی جریان را کاهش می‌دهد.

مزایای این تصمیم:

افزایش ایمنی تجهیزات

کاهش احتمال آسیب به مدارهای الکتریکی

افزایش طول عمر تجهیزات

کاهش احتمال بروز خطا هنگام راه‌اندازی مجدد سیستم

و معایب آن:

افزایش زمان لازم برای بازگشت کامل ساختمان به شرایط عادی

ایجاد تأخیر کوتاه در رسیدن دمای محیط به مقدار مطلوب

۴-۳. سند تصمیم‌گیری سوم (۰۳-ADR):

در این پروژه، تعداد زیادی حسگر و تجهیزات هوشمند به صورت مداوم اطلاعات مختلفی مانند دما، رطوبت، حضور افراد و وضعیت تجهیزات را برای بخش پردازش ارسال می‌کنند. بنابراین انتخاب پروتکل ارتباطی مناسب نقش بسیار مهمی در سرعت تبادل اطلاعات، میزان مصرف پهنای باند و عملکرد کلی سیستم دارد. در مرحله طراحی، استفاده از پروتکل HTTP نیز بررسی شد، اما به دلیل ماهیت درخواست و پاسخ (Request/Response)، سربار بیشتری به شبکه تحمیل می‌کرد و برای ارسال پیوسته داده‌های سنسوری گزینه مناسبی نبود.

تصمیم گرفته شده (Decision): پس از بررسی گزینه‌های موجود، پروتکل MQTT به عنوان پروتکل اصلی ارتباط میان تجهیزات اینترنت اشیا و گیت‌وی محلی انتخاب شد. این پروتکل از مدل Publish/Subscribe استفاده می‌کند و برای سیستم‌هایی که تعداد زیادی دستگاه به صورت هم‌زمان اطلاعات ارسال می‌کنند، گزینه بسیار مناسبی محسوب می‌شود. استفاده از MQTT باعث کاهش حجم پیام‌های ارسالی، افزایش سرعت تبادل اطلاعات و کاهش مصرف منابع سخت‌افزاری می‌شود. علاوه بر این، این پروتکل در بسیاری از سامانه‌های اینترنت اشیا مورد استفاده قرار گرفته و از نظر مقیاس‌پذیری نیز عملکرد مطلوبی دارد.

مزایای این تصمیم:

کاهش مصرف پهنای باند

سرعت بیشتر در انتقال اطلاعات

امکان مدیریت تعداد زیادی حسگر به صورت هم‌زمان

کاهش بار پردازشی گیت‌وی

مناسب بودن برای شبکه‌های با کیفیت متوسط یا ضعیف

معایب این تصمیم:

برخی تجهیزات قدیمی به صورت مستقیم از MQTT پشتیبانی نمی‌کنند.

در این شرایط لازم است از یک لایه واسط یا Gateway Bridge برای تبدیل پروتکل استفاده شود که تا حدودی پیچیدگی پیاده‌سازی را افزایش می‌دهد.

و در نهایت: تصمیمات معماری ثبت شده در این بخش، مسیر اصلی طراحی سیستم مدیریت هوشمند انرژی ساختمان را مشخص می‌کنند. هر یک از این تصمیمات با توجه به نیازمندی‌های عملکردی و غیرعملکردی پروژه و همچنین شرایط واقعی بهره‌برداری اتخاذ شده‌اند. استفاده از معماری مبتنی بر پردازش لبه، طراحی فرآیند راه‌اندازی ایمن پس از بازیابی برق و انتخاب پروتکل ارتباطی مناسب، در کنار یکدیگر باعث افزایش پایداری، امنیت، قابلیت توسعه و کارایی سامانه شده‌اند. مستندسازی این تصمیمات علاوه بر شفاف‌سازی روند طراحی، امکان توسعه و نگهداری بهتر سیستم در آینده را نیز فراهم می‌کند.

بخش ب) معماری خدمت گرا SOA :

با افزایش پیچیدگی سیستم‌های نرم‌افزاری، استفاده از معماری‌هایی که امکان توسعه، نگهداری و گسترش سیستم را ساده‌تر کنند، اهمیت زیادی پیدا کرده است. یکی از رویکردهای رایج در این زمینه، معماری خدمت گرا (Service-Oriented Architecture یا SOA) است. در این معماری، سیستم به جای آنکه به صورت یک نرم‌افزار یکپارچه و وابسته طراحی شود، از مجموعه‌ای از سرویس‌های مستقل تشکیل می‌شود که هر کدام مسئول انجام وظیفه مشخصی هستند و از طریق رابط‌های استاندارد با یکدیگر ارتباط برقرار می‌کنند.

استفاده از معماری SOA در پروژه «سیستم مدیریت هوشمند انرژی ساختمان‌ها» باعث می‌شود هر بخش از سیستم به صورت مستقل توسعه یابد، بدون آنکه تغییرات ایجاد شده در یک سرویس، باعث اختلال در عملکرد سایر بخش‌ها شود.

۱- شناسایی سرویس‌های بالقوه سیستم بر اساس اصول SOA :

پس از تحلیل فرآیندهای اصلی سیستم، مشخص شد که بسیاری از فعالیت‌های نرم‌افزار را می‌توان به صورت سرویس‌های مستقل پیاده‌سازی کرد. هر یک از این سرویس‌ها مسئول انجام یک وظیفه مشخص هستند و در صورت نیاز از طریق رابط‌های استاندارد با سایر سرویس‌ها تبادل اطلاعات انجام می‌دهند. این تفکیک وظایف باعث می‌شود وابستگی میان بخش‌های مختلف کاهش یافته و توسعه یا به‌روزرسانی هر قسمت بدون ایجاد تغییرات گسترده در کل سیستم امکان‌پذیر باشد.

۱-۱. سرویس مدیریت هویت و دسترسی:

اولین سرویس شناسایی‌شده، سرویس مدیریت هویت و دسترسی کاربران است. از آنجا که کاربران مختلفی مانند مدیر ساختمان، ساکنان و تکنسین نگهداری با سامانه در ارتباط هستند، لازم است قبل از دسترسی به امکانات سیستم، هویت آن‌ها بررسی و سطح دسترسی مناسب برای هر کاربر تعیین شود. این سرویس مسئول دریافت اطلاعات ورود کاربران، اعتبارسنجی نام کاربری و رمز عبور، احراز هویت و در نهایت صدور مجوز دسترسی متناسب با نقش هر فرد است. همچنین تمامی

درخواست‌های دسترسی ابتدا از این سرویس عبور می‌کنند تا از ورود افراد غیرمجاز و ایجاد تغییرات ناخواسته در تنظیمات سیستم جلوگیری شود. استقلال این سرویس از سایر بخش‌های نرم‌افزار باعث می‌شود در آینده بتوان روش‌های جدید احراز هویت، مانند احراز هویت دومرحله‌ای یا ورود از طریق سامانه‌های متمرکز، را بدون ایجاد تغییر در سایر سرویس‌ها به سیستم اضافه کرد.

۱-۲. سرویس جمع‌آوری و اعتبارسنجی داده‌های محیطی:

دومین سرویس، مسئول ارتباط مستقیم با تجهیزات اینترنت اشیا و دریافت اطلاعات محیطی است. این سرویس داده‌های ارسال‌شده توسط حسگرهای مختلف از جمله حسگر دما، رطوبت، نور و حضور افراد را دریافت کرده و قبل از ارسال آن‌ها به سایر بخش‌های سیستم، عملیات اولیه پردازش را انجام می‌دهد. در این مرحله، داده‌های نامعتبر، نویزهای احتمالی یا اطلاعات ناقص شناسایی شده و از ورود آن‌ها به فرآیند تصمیم‌گیری جلوگیری می‌شود. همچنین تمامی داده‌های معتبر همراه با زمان ثبت (Timestamp) در پایگاه داده ذخیره می‌شوند تا علاوه بر استفاده در تصمیم‌گیری لحظه‌ای، امکان تحلیل رفتار مصرف انرژی در بازه‌های زمانی مختلف نیز فراهم شود.

جدا بودن این سرویس از سایر بخش‌ها موجب می‌شود در صورت اضافه شدن حسگرهای جدید یا تغییر پروتکل ارتباطی تجهیزات، تنها این بخش نیاز به اصلاح داشته باشد و سایر سرویس‌های سیستم بدون تغییر به فعالیت خود ادامه دهند.

۱-۳. سرویس موتور قوانین و کنترل هوشمند:

اصلی‌ترین بخش تصمیم‌گیرنده سیستم، سرویس موتور قوانین و کنترل هوشمند است. این سرویس اطلاعات دریافتی از سرویس جمع‌آوری داده‌ها را دریافت کرده و آن‌ها را با قوانین و سیاست‌های تعریف‌شده توسط مدیر سیستم مقایسه می‌کند. در صورتی که شرایط لازم برای اجرای یک قانون برقرار باشد، این سرویس به صورت خودکار فرمان مناسب را برای تجهیزات مختلف صادر می‌کند. برای مثال، اگر حسگر حضور عدم حضور افراد را در یک فضای مشخص تشخیص دهد و هم‌زمان سیستم روشنایی یا سرمایش آن بخش فعال باشد، موتور قوانین می‌تواند دستور تغییر وضعیت تجهیزات به حالت کم‌مصرف یا خاموش را صادر کند.

تمرکز منطق تصمیم‌گیری در این سرویس باعث می‌شود قوانین کنترلی به صورت متمرکز مدیریت شوند و در صورت تغییر سیاست‌های مدیریت انرژی، تنها همین سرویس نیاز به به‌روزرسانی داشته باشد. به همین دلیل این سرویس یکی از مهم‌ترین اجزای معماری خدمت‌گرای سیستم محسوب می‌شود.

۱-۴. سرویس مدیریت هشدار و اعلان‌ها:

آخرین سرویس اصلی سیستم، سرویس مدیریت هشدارها و اعلان‌ها است. این سرویس به صورت مداوم وضعیت تجهیزات، اطلاعات دریافتی از حسگرها و نتایج پردازش سایر سرویس‌ها را پایش می‌کند تا در صورت مشاهده هرگونه وضعیت غیرعادی، اقدامات لازم را انجام دهد. در صورتی که خرابی تجهیزات، قطع ارتباط حسگرها، افزایش غیرعادی مصرف انرژی یا هر رویداد بحرانی دیگری شناسایی شود، این سرویس پیام هشدار مناسب را تولید کرده و آن را در اختیار کاربران مسئول، به‌ویژه مدیر ساختمان و تکنسین نگهداری، قرار می‌دهد. علاوه بر نمایش هشدار در داشبورد مدیریتی، این سرویس قابلیت ارسال اعلان به سایر کانال‌های ارتباطی سیستم را نیز خواهد داشت.

بررسی سرویس‌های شناسایی شده نشان می‌دهد که وظایف اصلی سیستم مدیریت هوشمند انرژی ساختمان‌ها به چهار سرویس مستقل تقسیم شده‌اند که هر یک مسئول انجام بخشی از فرآیند کلی سامانه هستند. این تفکیک علاوه بر کاهش وابستگی میان بخش‌های مختلف نرم‌افزار، توسعه، نگهداری و گسترش سیستم را نیز ساده‌تر می‌کند. همچنین استفاده از معماری خدمت‌گرا این امکان را فراهم می‌سازد که در آینده سرویس‌های جدید بدون ایجاد تغییرات اساسی در ساختار فعلی به سامانه اضافه شوند و سیستم با نیازهای جدید سازگار باقی بماند.

۲- طراحی رابط‌های سرویس و قراردادهای داده‌ای (Service Contracts)

پس از شناسایی سرویس‌های اصلی سیستم، لازم است نحوه تعامل میان این سرویس‌ها نیز به صورت مشخص تعریف شود. در معماری خدمت‌گرا (SOA)، ارتباط میان سرویس‌ها بر اساس مجموعه‌ای از قراردادهای مشخص انجام می‌شود که به آن Service Contract گفته می‌شود. این قراردادها تعیین می‌کنند که هر سرویس چه وظیفه‌ای بر عهده دارد، چه اطلاعاتی را دریافت می‌کند، چه خروجی‌ای تولید می‌کند و سایر سرویس‌ها تحت چه شرایطی می‌توانند از خدمات آن استفاده کنند. تعریف دقیق قراردادهای سرویس باعث کاهش وابستگی میان اجزای سیستم، افزایش قابلیت توسعه، سهولت نگهداری و جلوگیری از بروز ناسازگاری در هنگام توسعه نسخه‌های جدید نرم‌افزار می‌شود. به همین دلیل، برای هر یک از سرویس‌های اصلی سیستم مدیریت هوشمند انرژی ساختمان‌ها، یک قرارداد عملکردی در نظر گرفته شده است که در ادامه تشریح می‌شود.

۲-۱. قرارداد سرویس مدیریت هویت و دسترسی:

این سرویس مسئول کنترل هویت کاربران و مدیریت سطح دسترسی آن‌ها در سیستم است. تمامی کاربران پیش از استفاده از امکانات سامانه باید ابتدا از طریق این سرویس احراز هویت شوند.

هدف سرویس: تأیید هویت کاربران و تعیین سطح دسترسی متناسب با نقش هر کاربر.

ورودی‌های سرویس:

اطلاعات ورود کاربر (نام کاربری و رمز عبور)

درخواست ورود به سامانه

اطلاعات مربوط به نقش کاربر

خروجی‌های سرویس:

تأیید یا رد هویت کاربر

ایجاد نشست کاربری (Session)

اختصاص سطح دسترسی مناسب

ارسال مجوز استفاده از سایر سرویس‌ها

سرویس‌های وابسته:

سرویس موتور قوانین

سرویس مدیریت هشدار

داشبورد مدیریتی

شرایط اجرا: تنها کاربرانی که هویت آن‌ها با موفقیت تأیید شده باشد، مجاز به استفاده از سایر خدمات سامانه خواهند بود.

۲-۲. قرارداد سرویس جمع‌آوری و اعتبارسنجی داده‌های محیطی:

این سرویس نقطه ورود اطلاعات سنسورها به سیستم محسوب می‌شود و وظیفه دارد داده‌های محیطی را دریافت، بررسی و برای پردازش آماده کند.

هدف سرویس: دریافت اطلاعات خام تجهیزات اینترنت اشیا و تبدیل آن‌ها به داده‌های معتبر و قابل استفاده.

ورودی‌های سرویس:

اطلاعات حسگر دما

اطلاعات حسگر حضور

اطلاعات حسگر رطوبت

اطلاعات حسگر نور

زمان ثبت داده‌ها

خروجی‌های سرویس:

داده‌های اعتبارسنجی شده

حذف داده‌های نامعتبر یا نویزی

ذخیره اطلاعات در پایگاه داده

ارسال داده‌های معتبر به سرویس موتور قوانین

سرویس‌های وابسته:

سرویس موتور قوانین

پایگاه داده

سرویس مدیریت هشدار

شرایط اجرا: داده‌های دریافتی باید از نظر صحت، کامل بودن و بازه مجاز مقادیر بررسی شوند و تنها اطلاعات معتبر وارد

فرآیند تصمیم‌گیری شوند.

۲-۳. قرارداد سرویس موتور قوانین و کنترل هوشمند:

این سرویس هسته اصلی تصمیم‌گیری سیستم محسوب می‌شود و مسئول تحلیل اطلاعات محیطی و صدور فرمان‌های کنترلی است.

هدف سرویس: تحلیل شرایط ساختمان و اجرای قوانین مدیریت هوشمند انرژی.

ورودی‌های سرویس:

داده‌های اعتبارسنجی شده حسگرها

قوانین تعریف شده توسط مدیر ساختمان

وضعیت فعلی تجهیزات

خروجی‌های سرویس:

فرمان روشن یا خاموش شدن تجهیزات

تغییر وضعیت تجهیزات به حالت کم مصرف

ارسال فرمان تنظیم دمای سیستم HVAC

ثبت نتایج تصمیم‌گیری در پایگاه داده

سرویس‌های وابسته:

سرویس جمع‌آوری داده

سرویس مدیریت هشدار

عملگرهای هوشمند ساختمان

شرایط اجرا: تمام تصمیمات باید بر اساس قوانین تعریف شده سیستم و اطلاعات لحظه‌ای حسگرها انجام شوند و هیچ فرمانی

بدون بررسی شرایط محیط صادر نشود.

۲-۴. قرارداد سرویس مدیریت هشدار و اعلان‌ها:

این سرویس وظیفه پایش مداوم وضعیت سیستم و اطلاع‌رسانی درباره رخدادهای غیرعادی را بر عهده دارد.

هدف سرویس: تشخیص رخدادهای بحرانی و اطلاع‌رسانی سریع به کاربران مسئول.

ورودی‌های سرویس:

اطلاعات خرابی تجهیزات

گزارش خطاهای حسگرها

افزایش غیرعادی مصرف انرژی

پیام‌های تولید شده توسط موتور قوانین

خروجی‌های سرویس:

نمایش هشدار در داشبورد مدیریتی

ثبت رویداد در سیستم

ارسال اعلان برای مدیر ساختمان

ارسال هشدار برای تکنسین نگهداری

سرویس‌های وابسته:

موتور قوانین

داشبورد مدیریتی

پایگاه داده

شرایط اجرا: هشدارها باید بر اساس سطح اهمیت اولویت‌بندی شوند تا رویدادهای بحرانی سریع‌تر به کاربران مسئول اطلاع داده شوند.

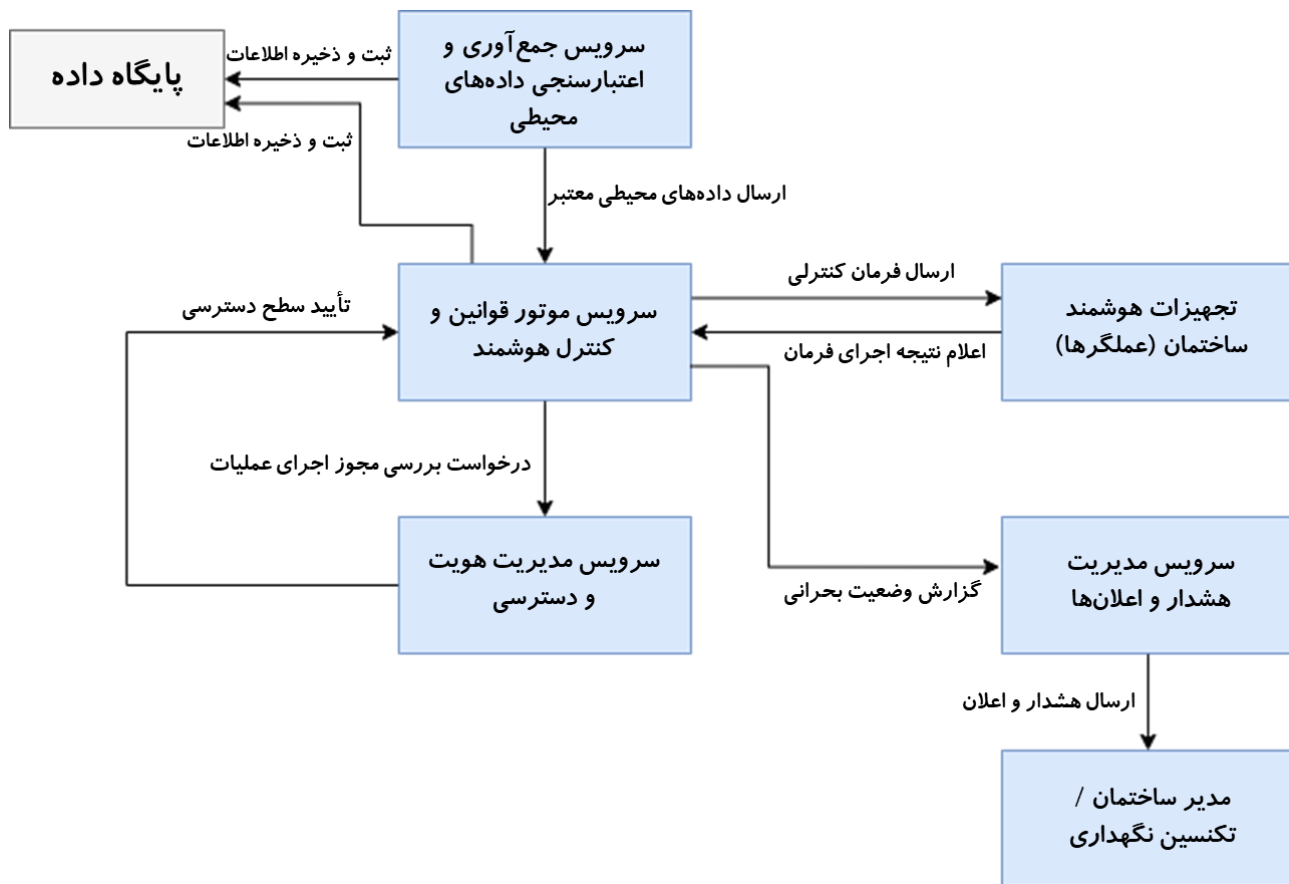
در نهایت، طراحی قراردادهای سرویس، یکی از مهم‌ترین مراحل معماری خدمت‌گرا محسوب می‌شود؛ زیرا نحوه تعامل میان سرویس‌های مختلف را به صورت شفاف مشخص می‌کند. در سیستم مدیریت هوشمند انرژی ساختمان‌ها، برای هر سرویس، وظایف، ورودی‌ها، خروجی‌ها، وابستگی‌ها و شرایط اجرای عملیات به طور مستقل تعریف شده است. این رویکرد باعث می‌شود هر سرویس بدون وابستگی مستقیم به پیاده‌سازی داخلی سایر سرویس‌ها فعالیت کند و در صورت توسعه یا به‌روزرسانی هر بخش، کمترین تأثیر بر سایر قسمت‌های سامانه ایجاد شود. همچنین وجود قراردادهای مشخص، فرآیند توسعه، آزمون، نگهداری و گسترش سیستم را در مراحل بعدی پروژه ساده‌تر و قابل مدیریت‌تر خواهد کرد.

۳- نمودار همکاری سرویس‌ها (Service Collaboration Diagram):

پس از شناسایی سرویس‌های اصلی و تعریف قراردادهای ارتباطی هر یک از آن‌ها، لازم است نحوه تعامل این سرویس‌ها در قالب یک سناریوی عملی نیز مشخص شود. اگرچه هر سرویس در معماری خدمت‌گرا وظیفه‌ای مستقل بر عهده دارد، اما تحقق اهداف اصلی سیستم تنها زمانی امکان‌پذیر است که این سرویس‌ها بتوانند به صورت هماهنگ و منظم با یکدیگر تبادل اطلاعات داشته باشند. به همین دلیل، در این بخش از نمودار همکاری سرویس‌ها (Service Collaboration Diagram) استفاده شده است. این نمودار، توالی ارتباط میان سرویس‌های اصلی سیستم مدیریت هوشمند انرژی ساختمان را در زمان وقوع یک رویداد واقعی نمایش می‌دهد. سناریوی انتخاب‌شده برای این نمودار، تشخیص تغییر شرایط محیطی و اجرای خودکار فرمان کنترلی است. در این فرآیند، داده‌های جمع‌آوری‌شده از حسگرها به تدریج میان سرویس‌های مختلف جابه‌جا شده و پس از انجام بررسی‌های لازم، فرمان مناسب برای تجهیزات ساختمان صادر می‌شود. همچنین در صورت بروز هرگونه خطا یا عدم موفقیت در اجرای فرمان، سیستم قادر خواهد بود هشدار لازم را برای مدیر ساختمان یا تکنسین نگهداری ارسال کند.

به طور کلی، این نمودار نشان می‌دهد که هیچ‌یک از سرویس‌ها به تنهایی مسئول اجرای کل فرآیند نیستند، بلکه هر سرویس تنها وظیفه تخصصی خود را انجام داده و نتیجه را در اختیار سرویس بعدی قرار می‌دهد. این شیوه تعامل، یکی از مهم‌ترین ویژگی‌های معماری خدمت‌گرا محسوب می‌شود و موجب کاهش وابستگی میان اجزای نرم‌افزار، افزایش قابلیت توسعه و ساده‌تر شدن نگهداری سیستم خواهد شد.

در شکل زیر نمودار را داریم:



۴- تحلیل مزایای SOA برای سیستم خاص (قابلیت استفاده مجدد، مقیاس پذیری، اتصال ضعیف):

تحلیل فرآیندی و مزایای راهبردی رویکرد SOA در سیستم مدیریت هوشمند انرژی:

بررسی جریان داده‌ها در نمودار تعامل سرویس‌ها (Service Interaction Diagram) نشان می‌دهد که استفاده از معماری سرویس‌گرا (SOA) در سیستم مدیریت هوشمند انرژی ساختمان، صرفاً یک انتخاب طراحی نرم‌افزاری نیست، بلکه یک رویکرد مهندسی ضروری برای مدیریت پیچیدگی در سامانه‌های سایبر-فیزیکی مبتنی بر اینترنت اشیاء (IoT) محسوب می‌شود. در چنین سیستم‌هایی، حجم بالای داده‌های محیطی، تنوع تجهیزات سخت‌افزاری و نیاز به تصمیم‌گیری بلادرنگ، ایجاب می‌کند که اجزای سیستم به صورت مستقل، ماژولار و با حداقل وابستگی طراحی شوند.

در این چارچوب، سه ویژگی کلیدی معماری SOA شامل قابلیت استفاده مجدد، مقیاس‌پذیری و اتصال ضعیف، نقش تعیین‌کننده‌ای در افزایش کارایی، پایداری و توسعه‌پذیری سیستم ایفا می‌کنند. در ادامه، هر یک از این ویژگی‌ها با توجه به سناریوی سیستم مدیریت هوشمند انرژی ساختمان تشریح می‌شود.

قابلیت استفاده مجدد:

در معماری‌های سنتی یکپارچه، منطق پردازش داده معمولاً به صورت مستقیم با لایه دریافت اطلاعات از سخت‌افزار ترکیب

می‌شود. این موضوع باعث می‌شود هرگونه تغییر در نوع سنسورها یا تجهیزات، مستلزم تغییر در هسته اصلی سیستم باشد که هم هزینه‌بر است و هم ریسک خطا را افزایش می‌دهد.

در مقابل، در معماری SOA طراحی شده برای این سیستم، «سرویس موتور قوانین و کنترل هوشمند» به صورت کاملاً مستقل از منابع تولید داده عمل می‌کند. این سرویس تنها بر اساس قراردادهای استاندارد داده‌ای ورودی را دریافت و پردازش می‌کند و هیچ وابستگی مستقیمی به نوع یا برند تجهیزات سخت‌افزاری ندارد.

این ویژگی در عمل مزایای مهمی به همراه دارد. برای مثال، در صورت توسعه ساختمان و اضافه شدن زون‌های جدید یا حتی تغییر کامل برند و تکنولوژی سنسورها، نیازی به بازنویسی منطق تصمیم‌گیری سیستم وجود ندارد. تنها کافی است سرویس مربوط به جمع‌آوری و اعتبارسنجی داده‌ها به‌روزرسانی شود. در نتیجه، بخش‌های مرکزی سیستم بدون تغییر باقی مانده و قابلیت استفاده مجدد از سرویس‌ها در سناریوهای مختلف تضمین می‌شود. این موضوع به‌طور مستقیم باعث کاهش هزینه نگهداری و افزایش عمر مفید نرم‌افزار می‌گردد.

مقیاس‌پذیری مستقل:

یکی از چالش‌های اساسی در سیستم‌های مبتنی بر اینترنت اشیاء، حجم بالای داده‌های ورودی و نرخ بالای تولید آن‌هاست. در سیستم مدیریت هوشمند انرژی ساختمان، سنسورها به‌صورت مداوم داده‌هایی مانند دما، رطوبت، روشنایی و حضور افراد را ارسال می‌کنند. این حجم از داده می‌تواند در زمان افزایش تعداد واحدهای ساختمانی یا توسعه سیستم، به یک گلوگاه پردازشی تبدیل شود.

در معماری SOA، هر سرویس به‌صورت مستقل قابل مقیاس‌دهی است. به عنوان نمونه، «سرویس جمع‌آوری و اعتبارسنجی داده‌های محیطی» بیشترین بار پردازشی سیستم را بر عهده دارد، زیرا به‌طور مداوم در حال دریافت، پاک‌سازی و فیلتر کردن داده‌های ورودی است. در صورت افزایش تعداد سنسورها یا افزایش نرخ ارسال داده، می‌توان این سرویس را به‌صورت افقی توسعه داد؛ بدون آنکه سایر سرویس‌ها مانند موتور قوانین، مدیریت هویت یا سیستم هشدار تحت تأثیر قرار گیرند. این تفکیک مقیاس‌پذیری باعث می‌شود منابع پردازشی دقیقاً در همان بخشی از سیستم تخصیص یابند که نیاز واقعی وجود دارد. به بیان دیگر، به جای افزایش یکپارچه و پرهزینه کل سیستم، تنها سرویس‌های تحت فشار توسعه داده می‌شوند. این ویژگی علاوه بر بهینه‌سازی مصرف منابع، موجب افزایش پایداری کلی سیستم در شرایط بار بالا می‌گردد.

اتصال ضعیف و تاب‌آوری سیستم:

در سیستم‌های پیچیده کنترلی، یکی از مهم‌ترین مسائل، جلوگیری از وابستگی شدید بین اجزای مختلف است. در صورتی که اجزای سیستم به‌صورت tightly coupled طراحی شوند، خرابی یک بخش می‌تواند منجر به اختلال کامل در عملکرد کل سیستم شود؛ پدیده‌ای که به آن Single Point of Failure گفته می‌شود.

در معماری SOA این مشکل با استفاده از اصل اتصال ضعیف (Loose Coupling) برطرف شده است. در این سیستم، ارتباط بین سرویس‌ها نه به‌صورت مستقیم و وابسته، بلکه از طریق پیام‌ها و قراردادهای استاندارد تعریف شده (Service Contracts) انجام می‌شود.

در نمودار تعاملات سیستم، مشاهده می‌شود که موتور قوانین به عنوان هسته مرکزی، با سرویس‌های جانبی مانند مدیریت

هویت، عملگرهای کنترلی و سیستم اعلان هشدار تنها از طریق پیام‌های استاندارد (بخش های ۲ تا ۵) در ارتباط است. این طراحی باعث می‌شود هر سرویس بتواند به صورت مستقل توسعه، اجرا یا حتی در صورت نیاز موقتاً غیرفعال شود، بدون آنکه عملکرد سایر بخش‌ها مختل گردد.

برای مثال، در صورتی که سرویس اعلان هشدار به دلیل اختلال در شبکه پیامکی یا مشکل ارتباطی در دسترس نباشد، سیستم کنترل انرژی همچنان به عملکرد خود ادامه می‌دهد. موتور قوانین همچنان داده‌های سنسورها را پردازش کرده و تصمیمات کنترلی مانند روشن یا خاموش کردن تجهیزات HVAC را اجرا می‌کند. این استقلال عملیاتی موجب افزایش تاب‌آوری سیستم و تضمین تداوم عملکرد در شرایط بحرانی می‌شود.

در واقع، این معماری باعث می‌شود سیستم حتی در شرایط قطعی اینترنت، اختلال بخشی از زیرساخت ارتباطی یا خرابی یک سرویس فرعی، همچنان هسته اصلی کنترل و بهینه‌سازی انرژی را حفظ کند.

بخش ج) مقایسه رویکردهای سنتی و مدرن:

در طراحی و پیاده‌سازی سامانه‌هایی نظیر سیستم مدیریت هوشمند انرژی ساختمان، انتخاب متدولوژی توسعه نرم‌افزار و معماری سیستم نقش تعیین‌کننده‌ای در موفقیت نهایی پروژه دارد. دلیل این اهمیت، ماهیت ترکیبی این سیستم‌ها است که هم‌زمان با داده‌های بلادرنگ محیطی، تجهیزات فیزیکی، و منطق تصمیم‌گیری نرم‌افزاری در تعامل هستند. در این بخش، با هدف تحلیل دقیق‌تر انتخاب‌های طراحی انجام‌شده در پروژه، مقایسه‌ای ساختاری میان رویکردهای سنتی و رویکردهای مدرن ارائه می‌گردد. این مقایسه نه صرفاً در سطح نظری، بلکه با تمرکز بر ویژگی‌های خاص سیستم مدیریت انرژی (مانند IoT بودن، حساسیت کنترلی، و نیاز به پردازش بلادرنگ) انجام شده است. جدول مقایسه رویکردهای سنتی و مدرن در سیستم مدیریت انرژی ساختمان:

معیار مقایسه	رویکرد سنتی	رویکرد مدرن	تحلیل و استدلال برای سیستم مدیریت هوشمند انرژی
مدل فرآیند توسعه	آبشاری (Waterfall) V-Model /	چابک (Agile/Scrum) /DevOps	با توجه به ماهیت پویا و وابسته به داده‌های واقعی در سیستم‌های IoT، نیازمندی‌ها در طول زمان تغییر می‌کنند. استفاده از مدل چابک امکان اصلاح تدریجی الگوریتم‌های کنترل انرژی را بر اساس داده‌های واقعی مصرف فراهم می‌کند، در حالی که مدل آبشاری در برابر تغییرات پس از طراحی اولیه انعطاف‌پذیری بسیار پایینی دارد.
معماری سیستم	یکپارچه	سرویس‌گرا (SOA) / میکروسرویس / ابری	در سیستم مدیریت انرژی، حجم بالای داده‌های سنسورها و نیاز به پردازش هم‌زمان باعث می‌شود معماری یکپارچه به گلوگاه عملکردی تبدیل شود. در مقابل، SOA امکان جداسازی سرویس‌های حیاتی مانند جمع‌آوری داده، تصمیم‌گیری و کنترل عملگرها را فراهم کرده و پایداری سیستم را در سطح عملیاتی افزایش می‌دهد.
مدیریت نیازمندی‌ها	ایستا و از پیش تعریف شده	پویا و تکاملی	نیازمندی‌های این سیستم وابسته به شرایط محیطی ساختمان، رفتار کاربران و شرایط فصلی هستند. بنابراین رویکرد تکاملی در مدیریت نیازمندی‌ها امکان بازتنظیم سیاست‌های مصرف انرژی (Energy Policies) را بدون بازطراحی کل سیستم فراهم می‌سازد.
تست و اعتبارسنجی	انتهای چرخه توسعه	پیوسته (Continuous Testing/CI)	شود. به HVAC در سیستم‌های کنترلی، تأخیر در کشف خطا می‌تواند منجر به رفتار نادرست در تجهیزات همین دلیل، تست پیوسته و خودکار قبل و بعد از استقرار هر تغییر، ریسک خرابی در سطح تجهیزات فیزیکی را به حداقل می‌رساند.
استقرار سیستم	یک مرحله‌ای (Big Bang Deployment)	تدریجی و مرحله‌ای	استقرار یک‌باره در سیستم‌های ساختمانی می‌تواند کل زیرساخت کنترلی را در معرض خطا قرار دهد. در مقابل، استقرار تدریجی اجازه می‌دهد سیستم ابتدا در یک ناحیه محدود (Zone-based Deployment) آزمایش شده و سپس به کل ساختمان تعمیم داده شود.
مدیریت تغییرات	رسمی، سنگین، زمان‌بر	سبک، انعطاف‌پذیر و سریع	در این پروژه، تغییر پروتکل‌های ارتباطی IoT یا جایگزینی سنسورها یک امر محتمل است. معماری مدرن با استفاده از لایه‌های انتزاعی (Abstraction Layers) اجازه می‌دهد تغییرات سخت‌افزاری بدون تغییر در منطق مرکزی سیستم اعمال شوند.

ضرورت استفاده از متدولوژی چابک:

سیستم مدیریت هوشمند انرژی ساختمان برخلاف نرم افزارهای کلاسیک، در یک محیط ایستا اجرا نمی شود، بلکه به طور مستقیم تحت تأثیر شرایط محیطی، رفتار کاربران و تغییرات بار مصرفی قرار دارد. این ویژگی باعث می شود فرض “ثبات نیازمندی ها” در مدل های سنتی مانند آبشاری، در این پروژه غیرواقعی باشد. در نتیجه، استفاده از متدولوژی چابک (Agile) این امکان را فراهم می کند که سیستم در چرخه های کوتاه توسعه به صورت مداوم ارزیابی و اصلاح شود. این موضوع به ویژه در تنظیم الگوریتم های کنترل انرژی اهمیت دارد، زیرا پارامترهایی مانند دمای بهینه، الگوی حضور کاربران و شرایط فصلی در طول زمان تغییر می کنند.

برتری معماری سرویس گرا (SOA) نسبت به معماری یکپارچه:

در معماری های یکپارچه، تمامی ماژول ها در یک ساختار واحد اجرا می شوند که این امر باعث ایجاد وابستگی شدید بین بخش های مختلف سیستم می گردد. در سیستم های IoT-محور، این وابستگی می تواند منجر به کاهش کارایی و افزایش ریسک خرابی زنجیره ای شود. در مقابل، معماری SOA با تفکیک سرویس ها به واحدهای مستقل، امکان جداسازی مسئولیت ها را فراهم می کند. برای مثال، سرویس جمع آوری داده های سنسورها می تواند به صورت مستقل از سرویس تصمیم گیری (Rule Engine) و سرویس کنترل عملگرها عمل کند. این جداسازی باعث می شود که افزایش بار در یک بخش، کل سیستم را تحت تأثیر قرار ندهد و پایداری کلی سیستم حفظ شود.

اهمیت تست و استقرار پیوسته در سیستم های کنترلی:

در سیستم های معمول نرم افزاری، خطاها ممکن است تنها منجر به اختلال عملکرد شوند، اما در سیستم های کنترلی مانند مدیریت انرژی ساختمان، خطا می تواند مستقیماً بر تجهیزات فیزیکی مانند سیستم های سرمایشی و گرمایشی تأثیر بگذارد. به همین دلیل، استفاده از CI/CD و تست های خودکار در این پروژه یک الزام مهندسی محسوب می شود، نه یک انتخاب. هر تغییر در منطق سیستم باید ابتدا در محیط بررسی شده و تنها پس از تأیید، به محیط واقعی ساختمان اعمال شود. این فرآیند ریسک عملیاتی را به حداقل رسانده و قابلیت اطمینان سیستم را افزایش می دهد. در مجموع، تحلیل انجام شده نشان می دهد که ماهیت پویا، توزیع شده و بلادرنگ سیستم مدیریت هوشمند انرژی ساختمان، استفاده از رویکردهای مدرن توسعه نرم افزار و معماری های سرویس گرا را اجتناب ناپذیر می سازد. رویکردهای سنتی به دلیل ساختار خطی و غیرقابل انعطاف، توان پاسخ گویی به پیچیدگی های این دسته از سیستم ها را ندارند.